Dokumentasi Resmi & Sintaks Go (Golang 1.22/1.23 Up-to-Date)

Bahasa: Go | Total Chunk/Dokumen: 114 | Disiapkan untuk RAG System

[golang_001] Go - Getting Started & Tooling

Tutorial: Get Started with Go (Installation & CLI): Prerequisites

Sumber: ht

Prerequisites
Some programming experience. The code here is pretty
simple, but it helps to know something about functions.
A tool to edit your code. Any text editor you have will
work fine. Most text editors have good support for Go. The most popular are
VSCode (free), GoLand (paid), and Vim (free).
A command terminal. Go works well using any terminal on
Linux and Mac, and on PowerShell or cmd in Windows.

[golang_002] Go - Getting Started & Tooling

Tutorial: Get Started with Go (Installation & CLI): Install Go

Sumber: ht

Install Go
Just use the Download and install steps.

[golang_003] Go - Getting Started & Tooling

Tutorial: Get Started with Go (Installation & CLI): Write some code

Sumber: ht

Write some code
Get started with Hello, World.
Open a command prompt and cd to your home directory.
On Linux or Mac:
cd
On Windows:
cd %HOMEPATH%
Create a hello directory for your first Go source code.
For example, use the following commands:
mkdir hello
cd hello
Enable dependency tracking for your code.
When your code imports packages contained in other modules, you manage
those dependencies through your code's own module. That module is defined
by a go.mod file that tracks the modules that provide those packages. That
go.mod file stays with your code, including in your source code
repository.
To enable dependency tracking for your code by creating a go.mod file, run
the
go mod init   command,
giving it the name of the module your code will be in. The name is the
module's module path.
In actual development, the module path will typically be the repository
location where your source code will be kept. For example, the module

path might be github.com/mymodule . If you plan to publish
your module for others to use, the module path must be a
location from which Go tools can download your module. For more about
naming a module with a module path, see
Managing
dependencies .
For the purposes of this tutorial, just use
example/hello .
$ go mod init example/hello
go: creating new go.mod: module example/hello
In your text editor, create a file hello.go in which to write your code.
Paste the following code into your hello.go file and save the file.
package main
import "fmt"
func main() {
fmt.Println("Hello, World!")
}
This is your Go code. In this code, you:
Declare a main package (a package is a way to group
functions, and it's made up of all the files in the same directory).
Import the popular
fmt package ,
which contains functions for formatting text, including printing

[golang_004] Go - Getting Started & Tooling

Tutorial: Get Started with Go (Installation & CLI): Call code in an external package

Sumber: ht

Call code in an external package
When you need your code to do something that might have been implemented by
someone else, you can look for a package that has functions you can use in
your code.
Make your printed message a little more interesting with a function from an
external module.
Visit pkg.go.dev and
search for a "quote" package .
In the search results, locate and click on the v1 of the rsc.io/quote package
(it should be listed with the "Other major versions" of rsc.io/quote/v4).
In the Documentation section, under Index , note the
list of functions you can call from your code. You'll use the
Go function.
At the top of this page, note that package quote is
included in the rsc.io/quote module.
You can use the pkg.go.dev site to find published modules whose packages
have functions you can use in your own code. Packages are published in
modules -- like rsc.io/quote -- where others can use them.
Modules are improved with new versions over time, and you can upgrade your
code to use the improved versions.
In your Go code, import the rsc.io/quote package and add a call
to its Go function.
After adding the highlighted lines, your code should include the
following:

Add new module requirements and sums.
Go will add the quote module as a requirement, as well as a
go.sum file for use in authenticating the module. For more, see
Authenticating modules in the Go
Modules Reference.
$ go mod tidy
go: finding module for package rsc.io/quote
go: found rsc.io/quote in rsc.io/quote v1.5.2
Run your code to see the message generated by the function you're calling.
$ go run .
Don't communicate by sharing memory, share memory by communicating.
Notice that your code calls the Go function, printing a
clever message about communication.
When you ran go mod tidy , it

Contoh Kode / Perintah CLI:

```
package main

import "fmt"

import "rsc.io/quote"

func main() {
    fmt.Println(quote.Go())
}
```

[golang_005] Go - Getting Started & Tooling

Tutorial: Get Started with Go (Installation & CLI): Write more code

Sumber: ht

Write more code
With this quick introduction, you got Go installed and learned some of the
basics. To write some more code with another tutorial, take a look at
Create a Go module .

[golang_006] Go - Go Modules & Dependency

Tutorial: Create a Go Module (go mod init, packages): Prerequisites

Sumber: ht

Prerequisites
Some programming experience. The code here is pretty
simple, but it helps to know something about functions, loops, and arrays.
Go. We recommend using the latest version of Go to follow this tutorial.
For installation instructions, see Installing Go .
A tool to edit your code. Any text editor you have will
work fine. Most text editors have good support for Go. The most popular are
VSCode (free), GoLand (paid), and Vim (free).
A command terminal. Go works well using any terminal on
Linux and Mac, and on PowerShell or cmd in Windows.

[golang_007] Go - Go Modules & Dependency

Tutorial: Create a Go Module (go mod init, packages): Start a module that others can use

Sumber: ht

Start a module that others can use
Start by creating a Go module. In a
module, you collect one or more related packages for a discrete and useful set
of functions. For example, you might create a module with packages that have
functions for doing financial analysis so that others writing financial
applications can use your work. For more about developing modules, see
Developing and publishing modules .
Go code is grouped into packages, and packages are grouped into modules. Your
module specifies dependencies needed to run your code, including the Go
version and the set of other modules it requires.
As you add or improve functionality in your module, you publish new versions
of the module. Developers writing code that calls functions in your module can
import the module's updated packages and test with the new version before
putting it into production use.
Open a command prompt and cd to your home directory.
On Linux or Mac:
cd
On Windows:
cd %HOMEPATH%
Create a greetings directory for your Go module source code.
For example, from your home directory use the following commands:
mkdir greetings
cd greetings
Start your module using the
go mod init command .
Run the go mod init command, giving it your module path --
here, use example.com/greetings . If you publish a module,
this must be a path from which your module can be downloaded by
Go tools. That would be your code's repository.
For more on naming your module with a module path, see
Managing
dependencies .
$ go mod init example.com/greetings
go: creating new go.mod: module example.com/greetings
The go mod init command creates a go.mod file to track your
code's dependencies. So far, the file includes only the name of your
module and the Go version your code supports. But as you add dependencies,
the go.mod f

[golang_008] Go - Generics (Type Parameters)

Tutorial: Getting Started with Generics (Type constraints, any): Overview

Sumber: ht

<!--{
"Title": "Tutorial: Getting started with generics",
"Breadcrumb": true
}-->
This tutorial introduces the basics of generics in Go. With generics, you can

declare and use functions or types that are written to work with any of a set
of types provided by calling code.
In this tutorial, you'll declare two simple non-generic functions, then capture
the same logic in a single generic function.
You'll progress through the following sections:
1. Create a folder for your code.
2. Add non-generic functions.
3. Add a generic function to handle multiple types.
4. Remove type arguments when calling the generic function.
5. Declare a type constraint.
Note: For other tutorials, see Tutorials.

[golang_009] Go - Generics (Type Parameters)

Tutorial: Getting Started with Generics (Type constraints, any): Prerequisites

Sumber: ht

Prerequisites
* **Go.** We recommend using the latest version of Go to follow this tutorial.
For installation instructions, see Installing Go.
* **A tool to edit your code.** Any text editor you have will work fine.
* **A command terminal.** Go works well using any terminal on Linux and Mac,
and on PowerShell or cmd in Windows.

[golang_010] Go - Generics (Type Parameters)

Tutorial: Getting Started with Generics (Type constraints, any): Create a folder for your code {#create_folder}

Sumber: ht

Create a folder for your code {#create_folder}
To begin, create a folder for the code you'll write.
1. Open a command prompt and change to your home directory.
On Linux or Mac:
On Windows:
The rest of the tutorial will show a $ as the prompt. The commands you use
will work on Windows too.
2. From the command prompt, create a directory for your code called generics.
3. Create a module to hold your code.
Run the `go mod init` command, giving it your new code's module path.
Note: For production code, you'd specify a module path that's more specific
to your own needs. For more, be sure to see
Managing dependencies.
Next, you'll add some simple code to work with maps.

Contoh Kode / Perintah CLI:

```
$ cd

// ----------------------------------------

C:\> cd %HOMEPATH%

// ----------------------------------------

$ mkdir generics
    $ cd generics
```

[golang_011] Go - Generics (Type Parameters)

Tutorial: Getting Started with Generics (Type constraints, any): Add non-generic functions {#non_generic_functions}

Sumber: ht

Add non-generic functions {#non_generic_functions}
In this step, you'll add two functions that each add together the values of a
map and return the total.
You're declaring two functions instead of one because you're working with two
different types of maps: one that stores `int64` values, and one that stores `float64` values.
Write the code
1. Using your text editor, create a file called main.go in the generics
directory. You'll write your Go code in this file.
2. Into main.go, at the top of the file, paste the following package
declaration.
A standalone program (as opposed to a library) is always in package `main`.
3. Beneath the package declaration, paste the following two function
declarations.
In this code, you:
* Declare two functions to add together the values of a map and return
the sum.
* `SumFloats` takes a map of `string` to `float64` values.
* `SumInts` takes a map of `string` to `int64` values.
4. At the top of main.go, beneath the package declaration, paste the following
`main` function to initialize the two maps and use them as arguments when
calling the functions you declared in the preceding step.
In this code, you:
* Initialize a map of `float64` values and a map of `int64` values, each
with two entries.
* Call the two functions you declared earlier to find the sum of each
map's values.
* Print the result.
5. Near the top of main.go, just beneath the package declaration, import the
package you'll need to support the code you've just written.
The first lines of code should look like this:
6. Save main.go.
Run the code
From the command line in the directory containing main.go, run the code.
With generics, you can write one function here instead of two. Next, you'll
add a single generic function for maps containing either integer o

Contoh Kode / Perintah CLI:

```
package main

// ----------------------------------------

// SumInts adds together the values of m.
    func SumInts(m map[string]int64) int64 {
     var s int64
     for _, v := range m {
      s += v
     }
     return s
    }
```

```
    // SumFloats adds together the values of m.
    func SumFloats(m map[string]float64) float64 {
     var s float64
     for _, v := range m {
      s += v
     }
     return s
    }

// ------------------------------------------

func main() {
     // Initialize a map for the integer values
     ints := map[string]int64{
      "first":  34,
      "second": 12,
     }

     // Initialize a map for the float values
     floats := map[string]float64{
      "first":  35.98,
      "second": 26.99,
     }

     fmt.Printf("Non-Generic Sums: %v and %v\n",
      SumInts(ints),
      SumFloats(floats))
    }
```

[golang_012] Go - Generics (Type Parameters)

Tutorial: Getting Started with Generics (Type constraints, any): Add a generic function to handle multiple types {#add_generic_function}

Sumber: ht

Add a generic function to handle multiple types {#add_generic_function}
In this section, you'll add a single generic function that can receive a map containing either integer or float values, effectively replacing the two functions you just wrote with a single function.
To support values of either type, that single function will need a way to declare what types it supports. Calling code, on the other hand, will need a way to specify whether it is calling with an integer or float map.
To support this, you'll write a function that declares _type parameters_ in addition to its ordinary function parameters. These type parameters make the function generic, enabling it to work with arguments of different types. You'll call the function with _type arguments_ and ordinary function arguments.
Each type parameter has a _type constraint_ that acts as a kind of meta-type for the type parameter. Each type constraint specifies the permissible type arguments that calling code can use for the respective type parameter.
While a type parameter's constraint typically represents a set of types, at compile time the type parameter stands for a single type - the type provided as a type argument by the calling code. If the type argument's type isn't allowed by the type parameter's constraint, the code won't compile.
Keep in mind that a type parameter must support all the operations the generic code is performing on it. For example, if your function's code were to try to perform `string` operations (such as indexing) on a type parameter whose constraint included numeric types, the code wouldn't compile.

In the code you're about to write, you'll use a constraint that allows either integer or float types.

Write the code

1. Beneath the two functions you added previously, paste the followin

Contoh Kode / Perintah CLI:

```
// SumIntsOrFloats sums the values of map m. It supports both int64 and float64
    // as types for map values.
    func SumIntsOrFloats[K comparable, V int64 | float64](m map[K]V) V {
        var s V
        for _, v := range m {
            s += v
        }
        return s
    }

// ----------------------------------------

fmt.Printf("Generic Sums: %v and %v\n",
     SumIntsOrFloats[string, int64](ints),
     SumIntsOrFloats[string, float64](floats))

// ----------------------------------------

$ go run .
Non-Generic Sums: 46 and 62.97
Generic Sums: 46 and 62.97
```

[golang_013] Go - Generics (Type Parameters)

Tutorial: Getting Started with Generics (Type constraints, any): Remove type arguments when calling the generic function {#remove_type_arguments}

Sumber: ht

Remove type arguments when calling the generic function {#remove_type_arguments}

In this section, you'll add a modified version of the generic function call, making a small change to simplify the calling code. You'll remove the type arguments, which aren't needed in this case.

You can omit type arguments in calling code when the Go compiler can infer the types you want to use. The compiler infers type arguments from the types of function arguments.

Note that this isn't always possible. For example, if you needed to call a generic function that had no arguments, you would need to include the type arguments in the function call.

Write the code

* In main.go, beneath the code you already have, paste the following code.

In this code, you:

* Call the generic function, omitting the type arguments.

Run the code

From the command line in the directory containing main.go, run the code.

Next, you'll further simplify the function by capturing the union of integers and floats into a type constraint you can reuse, such as from other code.

Contoh Kode / Perintah CLI:

```
fmt.Printf("Generic Sums, type parameters inferred: %v and %v\n",
     SumIntsOrFloats(ints),
     SumIntsOrFloats(floats))
```

```
// ----------------------------------------

$ go run .
Non-Generic Sums: 46 and 62.97
Generic Sums: 46 and 62.97
Generic Sums, type parameters inferred: 46 and 62.97
```

[golang_014] Go - Generics (Type Parameters)

Tutorial: Getting Started with Generics (Type constraints, any): Declare a type constraint {#declare_type_constraint}

Sumber: ht

Declare a type constraint {#declare_type_constraint}
In this last section, you'll move the constraint you defined earlier into its own interface so that you can reuse it in multiple places. Declaring constraints in this way helps streamline code, such as when a constraint is more complex.
You declare a _type constraint_ as an interface. The constraint allows any type implementing the interface. For example, if you declare a type constraint interface with three methods, then use it with a type parameter in a generic function, type arguments used to call the function must have all of those methods.
Constraint interfaces can also refer to specific types, as you'll see in this section.
Write the code
1. Just above `main`, immediately after the import statements, paste the following code to declare a type constraint.
In this code, you:
* Declare the `Number` interface type to use as a type constraint.
* Declare a union of `int64` and `float64` inside the interface.
Essentially, you're moving the union from the function declaration into a new type constraint. That way, when you want to constrain a type parameter to either `int64` or `float64`, you can use this `Number` type constraint instead of writing out `int64 | float64`.
2. Beneath the functions you already have, paste the following generic `SumNumbers` function.
In this code, you:
* Declare a generic function with the same logic as the generic function you declared previously, but with the new interface type instead of the union as the type constraint. As before, you use the type parameters for the argument and return types.
3. In main.go, beneath the code you already have, paste the following code.
In this code, you:
* Call `SumNumbers` with each map, printing the sum from the values of each.
As in the

Contoh Kode / Perintah CLI:

```
type Number interface {
        int64 | float64
    }

// ----------------------------------------

// SumNumbers sums the values of map m. It supports both integers
```

```
    // and floats as map values.
    func SumNumbers[K comparable, V Number](m map[K]V) V {
        var s V
        for _, v := range m {
            s += v
        }
        return s
    }

// ------------------------------------------

fmt.Printf("Generic Sums with Constraint: %v and %v\n",
     SumNumbers(ints),
     SumNumbers(floats))
```

[golang_015] Go - Generics (Type Parameters)

Tutorial: Getting Started with Generics (Type constraints, any): Conclusion {#conclusion}

Sumber: ht

Conclusion {#conclusion}
Nicely done! You've just introduced yourself to generics in Go.
Suggested next topics:
* The Go Tour is a great step-by-step
introduction to Go fundamentals.
* You'll find useful Go best practices described in
Effective Go and
How to write Go code.

[golang_016] Go - Generics (Type Parameters)

Tutorial: Getting Started with Generics (Type constraints, any): Completed code {#completed_code}

Sumber: ht

Completed code {#completed_code}
You can run this program in the
Go playground. On the
playground simply click the **Run** button.
```
package main
import "fmt"
type Number interface {
int64 | float64
}
func main() {
// Initialize a map for the integer values
ints := map[string]int64{
"first": 34,
"second": 12,
}
// Initialize a map for the float values
floats := map[string]float64{
"first": 35.98,
"second": 26.99,
```

```
}
fmt.Printf("Non-Generic Sums: %v and %v\n",
SumInts(ints),
SumFloats(floats))
fmt.Printf("Generic Sums: %v and %v\n",
SumIntsOrFloatsstring, int64,
SumIntsOrFloatsstring, float64)
fmt.Printf("Generic Sums, type parameters inferred: %v and %v\n",
SumIntsOrFloats(ints),
SumIntsOrFloats(floats))
fmt.Printf("Generic Sums with Constraint: %v and %v\n",
SumNumbers(ints),
SumNumbers(floats))
}
// SumInts adds together the values of m.
func SumInts(m map[string]int64) int64 {
var s int64
for _, v := range m {
s += v
}
return s
}
// SumFloats adds together the values of m.
func SumFloats(m map[string]float64) float64 {
var s float64
for _, v := range m {
s += v
}
return s
}
// SumIntsOrFloats sums the values of map m. It supports both floats and integers
// as map values.
func SumIntsOrFloatsK comparable, V int64 | float64 V {
var s V
for _, v := range m {
s += v
}
return s
}
// SumNumbers sums the values of map m. It supports both integers
// and floats as map values.
func SumNumbersK comparable, V Number V {
var s V
for _, v := range m {
s += v
}
return s
}
```

[golang_017] Go - Multi-Module Workspaces

Tutorial: Multi-Module Workspaces (go work init & go.work): Overview

Sumber: ht

<!--{
"Title": "Tutorial: Getting started with multi-module workspaces",
"Breadcrumb": true
}-->
This tutorial introduces the basics of multi-module workspaces in Go.
With multi-module workspaces, you can tell the Go command that you're
writing code in multiple modules at the same time and easily build and
run code in those modules.
In this tutorial, you'll create two modules in a shared multi-module
workspace, make changes across those modules, and see the results
of those changes in a build.
<!-- TODO TOC -->
Note: For other tutorials, see Tutorials.

[golang_018] Go - Multi-Module Workspaces

Tutorial: Multi-Module Workspaces (go work init & go.work): Prerequisites

Sumber: ht

Prerequisites
* **Go.** We recommend using the latest version of Go to follow this tutorial.
For installation instructions, see Installing Go.
* **A tool to edit your code.** Any text editor you have will work fine.
* **A command terminal.** Go works well using any terminal on Linux and Mac,
and on PowerShell or cmd in Windows.

[golang_019] Go - Multi-Module Workspaces

Tutorial: Multi-Module Workspaces (go work init & go.work): Create a module for your code {#create_folder}

Sumber: ht

Create a module for your code {#create_folder}
To begin, create a module for the code you'll write.
1. Open a command prompt and change to your home directory.
On Linux or Mac:
On Windows:
The rest of the tutorial will show a $ as the prompt. The commands you use
will work on Windows too.
2. From the command prompt, create a directory for your code called workspace.
3. Initialize the module
Our example will create a new module `hello` that will depend on the golang.org/x/example module.
Create the hello module:
Add a dependency on the golang.org/x/example/hello/reverse package by using `go get`.
Create hello.go in the hello directory with the following contents:
Now, run the hello program:

Contoh Kode / Perintah CLI:

```
$ cd

// ----------------------------------------

C:\> cd %HOMEPATH%
```

```
// ----------------------------------------

$ mkdir workspace
    $ cd workspace
```

[golang_020] Go - Multi-Module Workspaces

Tutorial: Multi-Module Workspaces (go work init & go.work): Create the workspace

Sumber: ht

Create the workspace
In this step, we'll create a `go.work` file to specify a workspace with the module.
Initialize the workspace
In the `workspace` directory, run:
The `go work init` command tells `go` to create a `go.work` file
for a workspace containing the modules in the `./hello`
directory.
The `go` command produces a `go.work` file that looks like this:
The `go.work` file has similar syntax to `go.mod`.
The `go` directive tells Go which version of Go the file should be
interpreted with. It's similar to the `go` directive in the `go.mod`
file.
The `use` directive tells Go that the module in the `hello`
directory should be main modules when doing a build.
So in any subdirectory of `workspace` the module will be active.
Run the program in the workspace directory
In the `workspace` directory, run:
The Go command includes all the modules in the workspace as main modules. This allows us to refer to a package in the module, even outside the module. Running the `go run` command outside the module or the workspace would result in an error because the `go` command wouldn't know which modules to use.
Next, we'll add a local copy of the `golang.org/x/example/hello` module to the workspace.
That module is stored in a subdirectory of the `go.googlesource.com/example` Git repository.
We'll then add a new function to the `reverse` package that we can use instead of `String`.

Contoh Kode / Perintah CLI:

```
$ go work init ./hello

// ----------------------------------------

go 1.18

   use ./hello

// ----------------------------------------

$ go run ./hello
   olleH
```

[golang_021] Go - Multi-Module Workspaces

Tutorial: Multi-Module Workspaces (go work init & go.work): Download and modify the `golang.org/x/example/hello` module

Sumber: ht

Download and modify the `golang.org/x/example/hello` module
In this step, we'll download a copy of the Git repo containing the `golang.org/x/example/hello` module,
add it to the workspace, and then add a new function to it that we will use from the hello program.
1. Clone the repository
From the workspace directory, run the `git` command to clone the repository:
2. Add the module to the workspace
The Git repo was just checked out into `./example`.
The source code for the `golang.org/x/example/hello` module is in `./example/hello`.
Add it to the workspace:
The `go work use` command adds a new module to the go.work file. It will now look like this:
The workspace now includes both the `example.com/hello` module and the `golang.org/x/example/hello` module,
which provides the `golang.org/x/example/hello/reverse` package.
This will allow us to use the new code we will write in our copy of the `reverse` package
instead of the version of the package in the module cache
that we downloaded with the `go get` command.
3. Add the new function.
We'll add a new function to reverse a number to the `golang.org/x/example/hello/reverse` package.
Create a new file named `int.go` in the `workspace/example/hello/reverse` directory containing the following contents:
4. Modify the hello program to use the function.
Modify the contents of `workspace/hello/hello.go` to contain the following contents:
Run the code in the workspace
From the workspace directory, run
The Go command finds the `example.com/hello` module specified in the
command line in the `hello` directory specified by the `go.work`
file, and similarly resolves the `golang.org/x/example/hello/reverse` import using
the `go.work` file.
`go.work` can be used instead of adding `replace`
directives to work across multiple modules.

Contoh Kode / Perintah CLI:

```
$ git clone https://go.googlesource.com/example
   Cloning into 'example'...
   remote: Total 165 (delta 27), reused 165 (delta 27)
   Receiving objects: 100% (165/165), 434.18 KiB | 1022.00 KiB/s, done.
   Resolving deltas: 100% (27/27), done.

// ----------------------------------------

$ go work use ./example/hello

// ----------------------------------------

go 1.18

   use (
       ./hello
       ./example/hello
   )
```

[golang_022] Go - Multi-Module Workspaces

Tutorial: Multi-Module Workspaces (go work init & go.work): Learn more about workspaces

Sumber: ht

Learn more about workspaces
The `go` command has a couple of subcommands for working with workspaces in addition to `go work init` which
we saw earlier in the tutorial:
- `go work use [-r] [dir]` adds a `use` directive to the `go.work` file for `dir`,
if it exists, and removes the `use` directory if the argument directory doesn't exist. The `-r`
flag examines subdirectories of `dir` recursively.
- `go work edit` edits the `go.work` file similarly to `go mod edit`
- `go work sync` syncs dependencies from the workspace's build list into each of the workspace modules.
See Workspaces in the Go Modules Reference for more detail on
workspaces and `go.work` files.

[golang_023] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Introduction

Sumber: ht

Introduction
Go is an open-source programming language that focuses on simplicity, reliability,
and efficiency, specifically designed to make it easy to build software at scale.
Although it borrows ideas from existing languages,
it has unusual properties that make effective Go programs
different in character from programs written in its relatives.
A straightforward translation of a C++ or Java program into Go
is unlikely to produce a satisfactory result—Java programs
are written in Java, not Go.
On the other hand, thinking about the problem from a Go
perspective could produce a successful but quite different
program.
In other words,
to write Go well, it's important to understand its properties
and idioms.
It's also important to know the established conventions for
programming in Go, such as naming, formatting, program
construction, and so on, so that programs you write
will be easy for other Go programmers to understand.
This document gives tips for writing clear, idiomatic Go code.
It augments the language specification ,
the Tour of Go ,
and How to Write Go Code ,
all of which you
should read first.

[golang_024] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Examples

Sumber: ht

Examples
The Go package sources
are intended to serve not
only as the core library but also as examples of how to
use the language.
Moreover, many of the packages contain working, self-contained
executable examples you can run directly from the
go.dev web site, such as
this one (if
necessary, click on the word "Example" to open it up).
If you have a question about how to approach a problem or how something
might be implemented, the documentation, code and examples in the
library can provide answers, ideas and
background.

[golang_025] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Formatting

Sumber: ht

Formatting
Formatting issues are the most contentious
but the least consequential.
People can adapt to different formatting styles
but it's better if they don't have to, and
less time is devoted to the topic
if everyone adheres to the same style.
The problem is how to approach this Utopia without a long
prescriptive style guide.
With Go we take an unusual
approach and let the machine
take care of most formatting issues.
The gofmt program
(also available as go fmt , which
operates at the package level rather than source file level)
reads a Go program
and emits the source in a standard style of indentation
and vertical alignment, retaining and if necessary
reformatting comments.
If you want to know how to handle some new layout
situation, run gofmt ; if the answer doesn't
seem right, rearrange your program (or file a bug about gofmt),
don't work around it.
As an example, there's no need to spend time lining up
the comments on the fields of a structure.
Gofmt will do that for you. Given the
declaration
gofmt will line up the columns:
All Go code in the standard packages has been formatted with gofmt .
Some formatting details remain. Very briefly:

Indentation
We use tabs for indentation and gofmt emits them by default.
Use spaces only if you must.
Line length
Go has no line length limit. Don't worry about overflowing a punched card.
If a line feels too long, wrap it and indent with an extra tab.
Parentheses
Go needs fewer parentheses than C and Java: control structures (if ,
for , switch) do not have parentheses in
their syntax.
Also, the operator precedence hierarchy is shorter and clearer, so
means what the spacing implies, unlike in the other languages.

Contoh Kode / Perintah CLI:

```
type T struct {
    name string // name of the object
    value int // its value
}

// ------------------------------------

type T struct {
    name    string // name of the object
    value   int    // its value
}

// ------------------------------------

x<<8 + y<<16
```

[golang_026] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Commentary

Sumber: ht

Commentary
Go provides C-style /* */ block comments
and C++-style // line comments.
Line comments are the norm;
block comments appear mostly as package comments, but
are useful within an expression or to disable large swaths of code.
Comments that appear before top-level declarations, with no intervening newlines,
are considered to document the declaration itself.
These "doc comments" are the primary documentation for a given Go package or command.
For more about doc comments, see " Go Doc Comments ".

[golang_027] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Names

Sumber: ht

Names
Names are as important in Go as in any other language.
They even have semantic effect:
the visibility of a name outside a package is determined by whether its
first character is upper case.
It's therefore worth spending a little time talking about naming conventions
in Go programs.

[golang_028] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Package names

Sumber: ht

Package names
When a package is imported, the package name becomes an accessor for the
contents. After
the importing package can talk about bytes.Buffer . It's
helpful if everyone using the package can use the same name to refer to
its contents, which implies that the package name should be good:
short, concise, evocative. By convention, packages are given
lower case, single-word names; there should be no need for underscores
or mixedCaps.
Err on the side of brevity, since everyone using your
package will be typing that name.
And don't worry about collisions a priori .
The package name is only the default name for imports; it need not be unique
across all source code, and in the rare case of a collision the
importing package can choose a different name to use locally.
In any case, confusion is rare because the file name in the import
determines just which package is being used.
Another convention is that the package name is the base name of
its source directory;
the package in src/encoding/base64
is imported as "encoding/base64" but has name base64 ,
not encoding_base64 and not encodingBase64 .
The importer of a package will use the name to refer to its contents,
so exported names in the package can use that fact
to avoid repetition.
(Don't use the import . notation, which can simplify
tests that must run outside the package they are testing, but should otherwise be avoided.)
For instance, the buffered reader type in the bufio package is called Reader ,
not BufReader , because users see it as bufio.Reader ,
which is a clear, concise name.
Moreover,
because imported entities are always addressed with their package name, bufio.Reader
does not conflict with io.Reader .
Similarly, the function to make new instances of ring.Ring —which
is the defi

Contoh Kode / Perintah CLI:

```
import "bytes"
```

[golang_029] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Getters

Sumber: ht

Getters
Go doesn't provide automatic support for getters and setters.
There's nothing wrong with providing getters and setters yourself,
and it's often appropriate to do so, but it's neither idiomatic nor necessary
to put Get into the getter's name. If you have a field called
owner (lower case, unexported), the getter method should be

called Owner (upper case, exported), not GetOwner .
The use of upper-case names for export provides the hook to discriminate
the field from the method.
A setter function, if needed, will likely be called SetOwner .
Both names read well in practice:

Contoh Kode / Perintah CLI:

```
owner := obj.Owner()
if owner != user {
    obj.SetOwner(user)
}
```

[golang_030] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Interface names

Sumber: ht

Interface names
By convention, one-method interfaces are named by
the method name plus an -er suffix or similar modification
to construct an agent noun: Reader ,
Writer , Formatter ,
CloseNotifier etc.
There are a number of such names and it's productive to honor them and the function
names they capture.
Read , Write , Close , Flush ,
String and so on have
canonical signatures and meanings. To avoid confusion,
don't give your method one of those names unless it
has the same signature and meaning.
Conversely, if your type implements a method with the
same meaning as a method on a well-known type,
give it the same name and signature;
call your string-converter method String not ToString .

[golang_031] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: MixedCaps

Sumber: ht

MixedCaps
Finally, the convention in Go is to use MixedCaps
or mixedCaps rather than underscores to write
multiword names.

[golang_032] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Semicolons

Sumber: ht

Semicolons
Like C, Go's formal grammar uses semicolons to terminate statements,
but unlike in C, those semicolons do not appear in the source.
Instead the lexer uses a simple rule to insert semicolons automatically
as it scans, so the input text is mostly free of them.
The rule is this. If the last token before a newline is an identifier
(which includes words like int and float64),

a basic literal such as a number or string constant, or one of the
tokens
the lexer always inserts a semicolon after the token.
This could be summarized as, “if the newline comes
after a token that could end a statement, insert a semicolon”.
A semicolon can also be omitted immediately before a closing brace,
so a statement such as
needs no semicolons.
Idiomatic Go programs have semicolons only in places such as
for loop clauses, to separate the initializer, condition, and
continuation elements. They are also necessary to separate multiple
statements on a line, should you write code that way.
One consequence of the semicolon insertion rules
is that you cannot put the opening brace of a
control structure (if , for , switch ,
or select) on the next line. If you do, a semicolon
will be inserted before the brace, which could cause unwanted
effects. Write them like this
not like this

Contoh Kode / Perintah CLI:

```
break continue fallthrough return ++ -- ) }

// ----------------------------------------

go func() { for { dst <- <-src } }()

// ----------------------------------------

if i < f() {
    g()
}
```

[golang_033] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Control structures

Sumber: ht

Control structures
The control structures of Go are related to those of C but differ
in important ways.
There is no do or while loop, only a
slightly generalized
for ;
switch is more flexible;
if and switch accept an optional
initialization statement like that of for ;
break and continue statements
take an optional label to identify what to break or continue;
and there are new control structures including a type switch and a
multiway communications multiplexer, select .
The syntax is also slightly different:
there are no parentheses
and the bodies must always be brace-delimited.

[golang_034] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: If

Sumber: ht

If
In Go a simple if looks like this:
Mandatory braces encourage writing simple if statements
on multiple lines. It's good style to do so anyway,
especially when the body contains a control statement such as a
return or break .
Since if and switch accept an initialization
statement, it's common to see one used to set up a local variable.
In the Go libraries, you'll find that
when an if statement doesn't flow into the next statement -- that is,
the body ends in break , continue ,
goto , or return -- the unnecessary
else is omitted.
This is an example of a common situation where code must guard against a
sequence of error conditions. The code reads well if the
successful flow of control runs down the page, eliminating error cases
as they arise. Since error cases tend to end in return
statements, the resulting code needs no else statements.

Contoh Kode / Perintah CLI:

```
if x > 0 {
    return y
}

// ----------------------------------------

if err := file.Chmod(0664); err != nil {
    log.Print(err)
    return err
}

// ----------------------------------------

f, err := os.Open(name)
if err != nil {
    return err
}
codeUsing(f)
```

[golang_035] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Redeclaration and reassignment

Sumber: ht

Redeclaration and reassignment
An aside: The last example in the previous section demonstrates a detail of how the
:= short declaration form works.
The declaration that calls os.Open reads,
This statement declares two variables, f and err .
A few lines later, the call to f.Stat reads,
which looks as if it declares d and err .
Notice, though, that err appears in both statements.

This duplication is legal: err is declared by the first statement,
but only re-assigned in the second.
This means that the call to f.Stat uses the existing
err variable declared above, and just gives it a new value.
In a := declaration a variable v may appear even
if it has already been declared, provided:
this declaration is in the same scope as the existing declaration of v
(if v is already declared in an outer scope, the declaration will create a new variable Section),
the corresponding value in the initialization is assignable to v , and
there is at least one other variable that is created by the declaration.
This unusual property is pure pragmatism,
making it easy to use a single err value, for example,
in a long if-else chain.
You'll see it used often.
Section It's worth noting here that in Go the scope of function parameters and return values
is the same as the function body, even though they appear lexically outside the braces
that enclose the body.

Contoh Kode / Perintah CLI:

```
f, err := os.Open(name)

// ----------------------------------------

d, err := f.Stat()
```

[golang_036] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: For

Sumber: ht

For
The Go for loop is similar to—but not the same as—C's.
It unifies for
and while and there is no do-while .
There are three forms, only one of which has semicolons.
Short declarations make it easy to declare the index variable right in the loop.
If you're looping over an array, slice, string, or map,
or reading from a channel, a range clause can
manage the loop.
If you only need the first item in the range (the key or index), drop the second:
If you only need the second item in the range (the value), use the blank identifier , an underscore, to discard the first:
The blank identifier has many uses, as described in a later section .
For strings, the range does more work for you, breaking out individual
Unicode code points by parsing the UTF-8.
Erroneous encodings consume one byte and produce the
replacement rune U+FFFD.
(The name (with associated builtin type) rune is Go terminology for a
single Unicode code point.
See the language specification
for details.)
The loop
prints
Finally, Go has no comma operator and ++ and --
are statements not expressions.
Thus if you want to run multiple variables in a for

you should use parallel assignment (although that precludes ++ and --).

Contoh Kode / Perintah CLI:

```
// Like a C for
for init; condition; post { }

// Like a C while
for condition { }

// Like a C for(;;)
for { }

// ----------------------------------------

sum := 0
for i := 0; i < 10; i++ {
    sum += i
}

// ----------------------------------------

for key, value := range oldMap {
    newMap[key] = value
}
```

[golang_037] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Switch

Sumber: ht

Switch
Go's switch is more general than C's.
The expressions need not be constants or even integers,
the cases are evaluated top to bottom until a match is found,
and if the switch has no expression it switches on
true .
It's therefore possible—and idiomatic—to write an
if - else - if - else
chain as a switch .
There is no automatic fall through, but cases can be presented
in comma-separated lists.
Although they are not nearly as common in Go as some other C-like
languages, break statements can be used to terminate
a switch early.
Sometimes, though, it's necessary to break out of a surrounding loop,
not the switch, and in Go that can be accomplished by putting a label
on the loop and "breaking" to that label.
This example shows both uses.
Of course, the continue statement also accepts an optional label
but it applies only to loops.
To close this section, here's a comparison routine for byte slices that uses two
switch statements:

Contoh Kode / Perintah CLI:

```
func unhex(c byte) byte {
    switch {
    case '0' <= c && c <= '9':
        return c - '0'
    case 'a' <= c && c <= 'f':
```

```
        return c - 'a' + 10
    case 'A' <= c && c <= 'F':
        return c - 'A' + 10
    }
    return 0
}

// --------------------------------------

func shouldEscape(c byte) bool {
    switch c {
    case ' ', '?', '&', '=', '#', '+', '%':
        return true
    }
    return false
}

// --------------------------------------

Loop:
    for n := 0; n < len(src); n += size {
        switch {
        case src[n] < sizeOne:
            if validateOnly {
                break
            }
            size = 1
            update(src[n])

        case src[n] < sizeTwo:
            if n+1 >= len(src) {
                err = errShortInput
                break Loop
            }
            if validateOnly {
                break
            }
            size = 2
            update(src[n] + src[n+1]<<shift)
        }
    }
```

[golang_038] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Type switch

Sumber: ht

Type switch
A switch can also be used to discover the dynamic type of an interface variable. Such a type switch uses the syntax of a type assertion with the keyword type inside the parentheses.
If the switch declares a variable in the expression, the variable will have the corresponding type in each clause.
It's also idiomatic to reuse the name in such cases, in effect declaring a new variable with the same name but a different type in each case.

Contoh Kode / Perintah CLI:

```
var t interface{}
t = functionOfSomeType()
switch t := t.(type) {
default:
```

```
    fmt.Printf("unexpected type %T\n", t)     // %T prints whatever type t has
case bool:
    fmt.Printf("boolean %t\n", t)             // t has type bool
case int:
    fmt.Printf("integer %d\n", t)             // t has type int
case *bool:
    fmt.Printf("pointer to boolean %t\n", *t) // t has type *bool
case *int:
    fmt.Printf("pointer to integer %d\n", *t) // t has type *int
}
```

[golang_039] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Multiple return values

Sumber: ht

Multiple return values
One of Go's unusual features is that functions and methods
can return multiple values. This form can be used to
improve on a couple of clumsy idioms in C programs: in-band
error returns such as -1 for EOF
and modifying an argument passed by address.
In C, a write error is signaled by a negative count with the
error code secreted away in a volatile location.
In Go, Write
can return a count and an error: “Yes, you wrote some
bytes but not all of them because you filled the device”.
The signature of the Write method on files from
package os is:
and as the documentation says, it returns the number of bytes
written and a non-nil error when n
!= len(b) .
This is a common style; see the section on error handling for more examples.
A similar approach obviates the need to pass a pointer to a return
value to simulate a reference parameter.
Here's a simple-minded function to
grab a number from a position in a byte slice, returning the number
and the next position.
You could use it to scan the numbers in an input slice b like this:

Contoh Kode / Perintah CLI:

```
func (file *File) Write(b []byte) (n int, err error)

// ----------------------------------------

func nextInt(b []byte, i int) (int, int) {
    for ; i < len(b) && !isDigit(b[i]); i++ {
    }
    x := 0
    for ; i < len(b) && isDigit(b[i]); i++ {
        x = x*10 + int(b[i]) - '0'
    }
    return x, i
}

// ----------------------------------------
```

```
for i := 0; i < len(b); {
        x, i = nextInt(b, i)
        fmt.Println(x)
    }
```

[golang_040] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Named result parameters

Sumber: ht

Named result parameters
The return or result "parameters" of a Go function can be given names and
used as regular variables, just like the incoming parameters.
When named, they are initialized to the zero values for their types when
the function begins; if the function executes a return statement
with no arguments, the current values of the result parameters are
used as the returned values.
The names are not mandatory but they can make code shorter and clearer:
they're documentation.
If we name the results of nextInt it becomes
obvious which returned int
is which.
Because named results are initialized and tied to an unadorned return, they can simplify
as well as clarify. Here's a version
of io.ReadFull that uses them well:

Contoh Kode / Perintah CLI:

```
func nextInt(b []byte, pos int) (value, nextPos int) {

// ----------------------------------------

func ReadFull(r Reader, buf []byte) (n int, err error) {
    for len(buf) > 0 && err == nil {
        var nr int
        nr, err = r.Read(buf)
        n += nr
        buf = buf[nr:]
    }
    return
}
```

[golang_041] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Defer

Sumber: ht

Defer
Go's defer statement schedules a function call (the
deferred function) to be run immediately before the function
executing the defer returns. It's an unusual but
effective way to deal with situations such as resources that must be
released regardless of which path a function takes to return. The
canonical examples are unlocking a mutex or closing a file.
Deferring a call to a function such as Close has two advantages. First, it
guarantees that you will never forget to close the file, a mistake
that's easy to make if you later edit the function to add a new return

path. Second, it means that the close sits near the open,
which is much clearer than placing it at the end of the function.
The arguments to the deferred function (which include the receiver if
the function is a method) are evaluated when the defer
executes, not when the call executes. Besides avoiding worries
about variables changing values as the function executes, this means
that a single deferred call site can defer multiple function
executions. Here's a silly example.
Deferred functions are executed in LIFO order, so this code will cause
4 3 2 1 0 to be printed when the function returns. A
more plausible example is a simple way to trace function execution
through the program. We could write a couple of simple tracing
routines like this:
We can do better by exploiting the fact that arguments to deferred
functions are evaluated when the defer executes. The
tracing routine can set up the argument to the untracing routine.
This example:
prints
For programmers accustomed to block-level resource management from
other languages, defer may seem peculiar, but its most
interesting and powerful applications come precisely from the fact
that it's not block-based but function-based. In th

Contoh Kode / Perintah CLI:

```
// Contents returns the file's contents as a string.
func Contents(filename string) (string, error) {
    f, err := os.Open(filename)
    if err != nil {
        return "", err
    }
    defer f.Close()  // f.Close will run when we're finished.

    var result []byte
    buf := make([]byte, 100)
    for {
        n, err := f.Read(buf[0:])
        result = append(result, buf[0:n]...) // append is discussed later.
        if err != nil {
            if err == io.EOF {
                break
            }
            return "", err  // f will be closed if we return here.
        }
    }
    return string(result), nil // f will be closed if we return here.
}

// ----------------------------------------

for i := 0; i < 5; i++ {
    defer fmt.Printf("%d ", i)
}

// ----------------------------------------

func trace(s string)   { fmt.Println("entering:", s) }
func untrace(s string) { fmt.Println("leaving:", s) }
```

```
// Use them like this:
func a() {
    trace("a")
    defer untrace("a")
    // do something....
}
```

[golang_042] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Allocation with new

Sumber: ht

Allocation with new
Go has two allocation primitives, the built-in functions
new and make .
They do different things and apply to different types, which can be confusing,
but the rules are simple.
Let's talk about new first.
It's a built-in function that allocates memory, but unlike its namesakes
in some other languages it does not initialize the memory,
it only zeros it.
That is,
new(T) allocates zeroed storage for a new variable of type
T and returns its address, a value of type *T .
In Go terminology, it returns a pointer to a newly allocated zero value of type
T .
Starting with Go 1.26, new also accepts an (value) expression as
an argument, which specifies the initial value of the variable.
For example, new(int64(300)) allocates a new variable of type int64 ,
initialized to 300, and returns its address.
Since the memory returned by new is zeroed, it's helpful to arrange
when designing your data structures that the
zero value of each type can be used without further initialization. This means a user of
the data structure can create one with new and get right to
work.
For example, the documentation for bytes.Buffer states that
"the zero value for Buffer is an empty buffer ready to use."
Similarly, sync.Mutex does not
have an explicit constructor or Init method.
Instead, the zero value for a sync.Mutex
is defined to be an unlocked mutex.
The zero-value-is-useful property works transitively. Consider this type declaration.
Values of type SyncedBuffer are also ready to use immediately upon allocation
or just declaration. In the next snippet, both p and v will work
correctly without further arrangement.

Contoh Kode / Perintah CLI:

```
type SyncedBuffer struct {
    lock    sync.Mutex
    buffer  bytes.Buffer
}

// ----------------------------------------

p := new(SyncedBuffer)  // type *SyncedBuffer
```

```
var v SyncedBuffer      // type  SyncedBuffer
```

[golang_043] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Constructors and composite literals

Sumber: ht

Constructors and composite literals
Sometimes the zero value isn't good enough and an initializing
constructor is necessary, as in this example derived from
package os .
There's a lot of boilerplate in there. We can simplify it
using a composite literal , which is
an expression that creates a
new instance each time it is evaluated.
Note that, unlike in C, it's perfectly OK to return the address of a local variable;
the storage associated with the variable survives after the function
returns.
In fact, taking the address of a composite literal
allocates a fresh instance each time it is evaluated,
so we can combine these last two lines.
The fields of a composite literal are laid out in order and must all be present.
However, by labeling the elements explicitly as field : value
pairs, the initializers can appear in any
order, with the missing ones left as their respective zero values. Thus we could say
As a limiting case, if a composite literal contains no fields at all, it creates
a zero value for the type. The expressions new(File) and &File{} are equivalent.
Composite literals can also be created for arrays, slices, and maps,
with the field labels being indices or map keys as appropriate.
In these examples, the initializations work regardless of the values of Enone ,
Eio , and Einval , as long as they are distinct.

Contoh Kode / Perintah CLI:

```
func NewFile(fd int, name string) *File {
    if fd < 0 {
        return nil
    }
    f := new(File)
    f.fd = fd
    f.name = name
    f.dirinfo = nil
    f.nepipe = 0
    return f
}

// ----------------------------------------

func NewFile(fd int, name string) *File {
    if fd < 0 {
        return nil
    }
    f := File{fd, name, nil, 0}
    return &f
}

// ----------------------------------------
```

```
return &File{fd, name, nil, 0}
```

[golang_044] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Allocation with make

Sumber: ht

Allocation with make
Back to allocation.
The built-in function make(T, args) serves
a purpose different from new(T) .
It creates slices, maps, and channels only, and it returns an initialized
(not zeroed)
value of type T (not *T).
The reason for the distinction
is that these three types represent, under the covers, references to data structures that
must be initialized before use.
A slice, for example, is a three-item descriptor
containing a pointer to the data (inside an array), the length, and the
capacity, and until those items are initialized, the slice is nil .
For slices, maps, and channels,
make initializes the internal data structure and prepares
the value for use.
For instance,
allocates an array of 100 ints and then creates a slice
structure with length 10 and a capacity of 100 pointing at the first
10 elements of the array.
(When making a slice, the capacity can be omitted; see the section on slices
for more information.)
In contrast, new([]int) returns a pointer to a newly allocated, zeroed slice
structure, that is, a pointer to a nil slice value.
These examples illustrate the difference between new and
make .
Remember that make applies only to maps, slices and channels
and does not return a pointer.
To obtain an explicit pointer allocate with new or take the address
of a variable explicitly.

Contoh Kode / Perintah CLI:

```
make([]int, 10, 100)

// ---------------------------------------

var p *[]int = new([]int)       // allocates slice structure; *p == nil; rarely useful
var v  []int = make([]int, 100) // the slice v now refers to a new array of 100 ints

// Unnecessarily complex:
var p *[]int = new([]int)
*p = make([]int, 100, 100)

// Idiomatic:
v := make([]int, 100)
```

[golang_045] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Arrays

Sumber: ht

Arrays
Arrays are useful when planning the detailed layout of memory and sometimes
can help avoid allocation, but primarily
they are a building block for slices, the subject of the next section.
To lay the foundation for that topic, here are a few words about arrays.
There are major differences between the ways arrays work in Go and C.
In Go,
Arrays are values. Assigning one array to another copies all the elements.
In particular, if you pass an array to a function, it
will receive a copy of the array, not a pointer to it.
The size of an array is part of its type. The types [10]int
and [20]int are distinct.
The value property can be useful but also expensive; if you want C-like behavior and efficiency,
you can pass a pointer to the array.
But even this style isn't idiomatic Go.
Use slices instead.

Contoh Kode / Perintah CLI:

```
func Sum(a *[3]float64) (sum float64) {
    for _, v := range *a {
        sum += v
    }
    return
}

array := [...]float64{7.0, 8.5, 9.1}
x := Sum(&array)  // Note the explicit address-of operator
```

[golang_046] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Slices

Sumber: ht

Slices
Slices wrap arrays to give a more general, powerful, and convenient
interface to sequences of data. Except for items with explicit
dimension such as transformation matrices, most array programming in
Go is done with slices rather than simple arrays.
Slices hold references to an underlying array, and if you assign one
slice to another, both refer to the same array.
If a function takes a slice argument, changes it makes to
the elements of the slice will be visible to the caller, analogous to
passing a pointer to the underlying array. A Read
function can therefore accept a slice argument rather than a pointer
and a count; the length within the slice sets an upper
limit of how much data to read. Here is the signature of the
Read method of the File type in package
os :
The method returns the number of bytes read and an error value, if
any.
To read into the first 32 bytes of a larger buffer
buf , slice (here used as a verb) the buffer.

Such slicing is common and efficient. In fact, leaving efficiency aside for the moment, the following snippet would also read the first 32 bytes of the buffer. The length of a slice may be changed as long as it still fits within the limits of the underlying array; just assign it to a slice of itself. The capacity of a slice, accessible by the built-in function cap , reports the maximum length the slice may assume. Here is a function to append data to a slice. If the data exceeds the capacity, the slice is reallocated. The resulting slice is returned. The function uses the fact that len and cap are legal when applied to the nil slice, and return 0.
We must return the slice afterwards because, although Append can modify the elements of slice , the slice itself (the run-time data structure holding the pointer, length

Contoh Kode / Perintah CLI:

```
func (f *File) Read(buf []byte) (n int, err error)

// ----------------------------------------

n, err := f.Read(buf[0:32])

// ----------------------------------------

var n int
    var err error
    for i := 0; i < 32; i++ {
        nbytes, e := f.Read(buf[i:i+1])  // Read one byte.
        n += nbytes
        if nbytes == 0 || e != nil {
            err = e
            break
        }
    }
```

[golang_047] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Two-dimensional slices

Sumber: ht

Two-dimensional slices
Go's arrays and slices are one-dimensional.
To create the equivalent of a 2D array or slice, it is necessary to define an array-of-arrays or slice-of-slices, like this:
Because slices are variable-length, it is possible to have each inner slice be a different length.
That can be a common situation, as in our LinesOfText example: each line has an independent length.
Sometimes it's necessary to allocate a 2D slice, a situation that can arise when processing scan lines of pixels, for instance.
There are two ways to achieve this.
One is to allocate each slice independently; the other is to allocate a single array and point the individual slices into it.
Which to use depends on your application.
If the slices might grow or shrink, they should be allocated independently

to avoid overwriting the next line; if not, it can be more efficient to construct
the object with a single allocation.
For reference, here are sketches of the two methods.
First, a line at a time:
And now as one allocation, sliced into lines:

Contoh Kode / Perintah CLI:

```
type Transform [3][3]float64  // A 3x3 array, really an array of arrays.
type LinesOfText [][]byte     // A slice of byte slices.

// ----------------------------------------

text := LinesOfText{
    []byte("Now is the time"),
    []byte("for all good gophers"),
    []byte("to bring some fun to the party."),
}

// ----------------------------------------

// Allocate the top-level slice.
picture := make([][]uint8, YSize) // One row per unit of y.
// Loop over the rows, allocating the slice for each row.
for i := range picture {
    picture[i] = make([]uint8, XSize)
}
```

[golang_048] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Maps

Sumber: ht

Maps
Maps are a convenient and powerful built-in data structure that associate
values of one type (the key) with values of another type
(the element or value).
The key can be of any type for which the equality operator is defined,
such as integers,
floating point and complex numbers,
strings, pointers, interfaces (as long as the dynamic type
supports equality), structs and arrays.
Slices cannot be used as map keys,
because equality is not defined on them.
Like slices, maps hold references to an underlying data structure.
If you pass a map to a function
that changes the contents of the map, the changes will be visible
in the caller.
Maps can be constructed using the usual composite literal syntax
with colon-separated key-value pairs,
so it's easy to build them during initialization.
Assigning and fetching map values looks syntactically just like
doing the same for arrays and slices except that the index doesn't
need to be an integer.
An attempt to fetch a map value with a key that
is not present in the map will return the zero value for the type
of the entries
in the map. For instance, if the map contains integers, looking

up a non-existent key will return 0 .
A set can be implemented as a map with value type bool .
Set the map entry to true to put the value in the set, and then
test it by simple indexing.
Sometimes you need to distinguish a missing entry from
a zero value. Is there an entry for "UTC"
or is that 0 because it's not in the map at all?
You can discriminate with a form of multiple assignment.
For obvious reasons this is called the “comma ok” idiom.
In this example, if tz is present, seconds
will be set appropriately and ok will be true; if not,
seconds will be set to zero and ok will
be false.
Here's a function that puts it t

Contoh Kode / Perintah CLI:

```
var timeZone = map[string]int{
    "UTC":  0*60*60,
    "EST": -5*60*60,
    "CST": -6*60*60,
    "MST": -7*60*60,
    "PST": -8*60*60,
}

// ----------------------------------------

offset := timeZone["EST"]

// ----------------------------------------

attended := map[string]bool{
    "Ann": true,
    "Joe": true,
    ...
}

if attended[person] { // will be false if person is not in the map
    fmt.Println(person, "was at the meeting")
}
```

[golang_049] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Printing

Sumber: ht

Printing
Formatted printing in Go uses a style similar to C's printf
family but is richer and more general. The functions live in the fmt
package and have capitalized names: fmt.Printf , fmt.Fprintf ,
fmt.Sprintf and so on. The string functions (Sprintf etc.)
return a string rather than filling in a provided buffer.
You don't need to provide a format string. For each of Printf ,
Fprintf and Sprintf there is another pair
of functions, for instance Print and Println .
These functions do not take a format string but instead generate a default
format for each argument. The Println versions also insert a blank
between arguments and append a newline to the output while

the Print versions add blanks only if the operand on neither side is a string.
In this example each line produces the same output.
The formatted print functions fmt.Fprint
and friends take as a first argument any object
that implements the io.Writer interface; the variables os.Stdout
and os.Stderr are familiar instances.
Here things start to diverge from C. First, the numeric formats such as %d
do not take flags for signedness or size; instead, the printing routines use the
type of the argument to decide these properties.
prints
If you just want the default conversion, such as decimal for integers, you can use
the catchall format %v (for “value”); the result is exactly
what Print and Println would produce.
Moreover, that format can print any value, even arrays, slices, structs, and
maps. Here is a print statement for the time zone map defined in the previous section.
which gives output:
For maps, Printf and friends sort the output lexicographically by key.
When printing a struct, the modified format %+v annotates the
fields of the structure with their names,

Contoh Kode / Perintah CLI:

```
fmt.Printf("Hello %d\n", 23)
fmt.Fprint(os.Stdout, "Hello ", 23, "\n")
fmt.Println("Hello", 23)
fmt.Println(fmt.Sprint("Hello ", 23))

// ----------------------------------------

var x uint64 = 1<<64 - 1
fmt.Printf("%d %x; %d %x\n", x, x, int64(x), int64(x))

// ----------------------------------------

18446744073709551615 ffffffffffffffff; -1 -1
```

[golang_050] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Append

Sumber: ht

Append
Now we have the missing piece we needed to explain the design of
the append built-in function. The signature of append
is different from our custom Append function above.
Schematically, it's like this:
where T is a placeholder for any given type. You can't
actually write a function in Go where the type T
is determined by the caller.
That's why append is built in: it needs support from the
compiler.
What append does is append the elements to the end of
the slice and return the result. The result needs to be returned
because, as with our hand-written Append , the underlying
array may change. This simple example
prints [1 2 3 4 5 6] . So append works a
little like Printf , collecting an arbitrary number of

arguments.
But what if we wanted to do what our Append does and
append a slice to a slice? Easy: use ... at the call
site, just as we did in the call to Output above. This
snippet produces identical output to the one above.
Without that ... , it wouldn't compile because the types
would be wrong; y is not of type int .

Contoh Kode / Perintah CLI:

```
func append(slice []T, elements ...T) []T

// ----------------------------------------

x := []int{1,2,3}
x = append(x, 4, 5, 6)
fmt.Println(x)

// ----------------------------------------

x := []int{1,2,3}
y := []int{4,5,6}
x = append(x, y...)
fmt.Println(x)
```

[golang_051] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Initialization

Sumber: ht

Initialization
Although it doesn't look superficially very different from
initialization in C or C++, initialization in Go is more powerful.
Complex structures can be built during initialization and the ordering
issues among initialized objects, even among different packages, are handled
correctly.

[golang_052] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Constants

Sumber: ht

Constants
Constants in Go are just that—constant.
They are created at compile time, even when defined as
locals in functions,
and can only be numbers, characters (runes), strings or booleans.
Because of the compile-time restriction, the expressions
that define them must be constant expressions,
evaluatable by the compiler. For instance,
1<<3 is a constant expression, while
math.Sin(math.Pi/4) is not because
the function call to math.Sin needs
to happen at run time.
In Go, enumerated constants are created using the iota
enumerator. Since iota can be part of an expression and
expressions can be implicitly repeated, it is easy to build intricate
sets of values.

{{code "/doc/progs/eff_bytesize.go" `/^type ByteSize/` `/^\)/`}}
The ability to attach a method such as String to any
user-defined type makes it possible for arbitrary values to format themselves
automatically for printing.
Although you'll see it most often applied to structs, this technique is also useful for
scalar types such as floating-point types like ByteSize .
{{code "/doc/progs/eff_bytesize.go" `/^func.*ByteSize.*String/` `/^}/`}}
The expression YB prints as 1.00YB ,
while ByteSize(1e13) prints as 9.09TB .
The use here of Sprintf
to implement ByteSize 's String method is safe
(avoids recurring indefinitely) not because of a conversion but
because it calls Sprintf with %f ,
which is not a string format: Sprintf will only call
the String method when it wants a string, and %f
wants a floating-point value.

[golang_053] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Variables

Sumber: ht

Variables
Variables can be initialized just like constants but the
initializer can be a general expression computed at run time.

Contoh Kode / Perintah CLI:

```
var (
    home   = os.Getenv("HOME")
    user   = os.Getenv("USER")
    gopath = os.Getenv("GOPATH")
)
```

[golang_054] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: The init function

Sumber: ht

The init function
Finally, each source file can define its own niladic init function to
set up whatever state is required. (Actually each file can have multiple
init functions.)
And finally means finally: init is called after all the
variable declarations in the package have evaluated their initializers,
and those are evaluated only after all the imported packages have been
initialized.
Besides initializations that cannot be expressed as declarations,
a common use of init functions is to verify or repair
correctness of the program state before real execution begins.

Contoh Kode / Perintah CLI:

```
func init() {
    if user == "" {
        log.Fatal("$USER not set")
    }
    if home == "" {
        home = "/home/" + user
    }
```

```
    if gopath == "" {
        gopath = home + "/go"
    }
    // gopath may be overridden by --gopath flag on command line.
    flag.StringVar(&gopath, "gopath", gopath, "override default GOPATH")
}
```

[golang_055] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Pointers vs. Values

Sumber: ht

Pointers vs. Values
As we saw with ByteSize ,
methods can be defined for any named type (except a pointer or an interface);
the receiver does not have to be a struct.
In the discussion of slices above, we wrote an Append
function. We can define it as a method on slices instead. To do
this, we first declare a named type to which we can bind the method, and
then make the receiver for the method a value of that type.
This still requires the method to return the updated slice. We can
eliminate that clumsiness by redefining the method to take a
pointer to a ByteSlice as its receiver, so the
method can overwrite the caller's slice.
In fact, we can do even better. If we modify our function so it looks
like a standard Write method, like this,
then the type *ByteSlice satisfies the standard interface
io.Writer , which is handy. For instance, we can
print into one.
We pass the address of a ByteSlice
because only *ByteSlice satisfies io.Writer .
The rule about pointers vs. values for receivers is that value methods
can be invoked on pointers and values, but pointer methods can only be
invoked on pointers.
This rule arises because pointer methods can modify the receiver; invoking
them on a value would cause the method to receive a copy of the value, so
any modifications would be discarded.
The language therefore disallows this mistake.
There is a handy exception, though. When the value is addressable, the
language takes care of the common case of invoking a pointer method on a
value by inserting the address operator automatically.
In our example, the variable b is addressable, so we can call
its Write method with just b.Write . The compiler
will rewrite that to (&b).Write for us.
By the way, the idea of using Write on a slice of bytes
is central to the im

Contoh Kode / Perintah CLI:

```
type ByteSlice []byte

func (slice ByteSlice) Append(data []byte) []byte {
    // Body exactly the same as the Append function defined above.
}

// ----------------------------------------
```

```
func (p *ByteSlice) Append(data []byte) {
    slice := *p
    // Body as above, without the return.
    *p = slice
}

// ---------------------------------------

func (p *ByteSlice) Write(data []byte) (n int, err error) {
    slice := *p
    // Again as above.
    *p = slice
    return len(data), nil
}
```

[golang_056] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Interfaces

Sumber: ht

Interfaces
Interfaces in Go provide a way to specify the behavior of an
object: if something can do this , then it can be used
here . We've seen a couple of simple examples already;
custom printers can be implemented by a String method
while Fprintf can generate output to anything
with a Write method.
Interfaces with only one or two methods are common in Go code, and are
usually given a name derived from the method, such as io.Writer
for something that implements Write .
A type can implement multiple interfaces.
For instance, a collection can be sorted
by the routines in package sort if it implements
sort.Interface , which contains Len() ,
Less(i, j int) bool , and Swap(i, j int) ,
and it could also have a custom formatter.
In this contrived example Sequence satisfies both.
{{code "/doc/progs/eff_sequence.go" `/^type/` "$"}}

[golang_057] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Conversions

Sumber: ht

Conversions
The String method of Sequence is recreating the
work that Sprint already does for slices.
(It also has complexity O(N^2), which is poor.) We can share the
effort (and also speed it up) if we convert the Sequence to a plain
[]int before calling Sprint .
This method is another example of the conversion technique for calling
Sprintf safely from a String method.
Because the two types (Sequence and []int)
are the same if we ignore the type name, it's legal to convert between them.
The conversion doesn't create a new value, it just temporarily acts
as though the existing value has a new type.

(There are other legal conversions, such as from integer to floating point, that
do create a new value.)
It's an idiom in Go programs to convert the
type of an expression to access a different
set of methods. As an example, we could use the existing
type sort.IntSlice to reduce the entire example
to this:
Now, instead of having Sequence implement multiple
interfaces (sorting and printing), we're using the ability of a data item to be
converted to multiple types (Sequence , sort.IntSlice
and []int), each of which does some part of the job.
That's more unusual in practice but can be effective.

Contoh Kode / Perintah CLI:

```
func (s Sequence) String() string {
    s = s.Copy()
    sort.Sort(s)
    return fmt.Sprint([]int(s))
}

// -----------------------------------------

type Sequence []int

// Method for printing - sorts the elements before printing
func (s Sequence) String() string {
    s = s.Copy()
    sort.IntSlice(s).Sort()
    return fmt.Sprint([]int(s))
}
```

[golang_058] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Interface conversions and type assertions

Sumber: ht

Interface conversions and type assertions
Type switches are a form of conversion: they take an interface and, for
each case in the switch, in a sense convert it to the type of that case.
Here's a simplified version of how the code under fmt.Printf turns a value into
a string using a type switch.
If it's already a string, we want the actual string value held by the interface, while if it has a
String method we want the result of calling the method.
The first case finds a concrete value; the second converts the interface into another interface.
It's perfectly fine to mix types this way.
What if there's only one type we care about? If we know the value holds a string
and we just want to extract it?
A one-case type switch would do, but so would a type assertion .
A type assertion takes an interface value and extracts from it a value of the specified explicit type.
The syntax borrows from the clause opening a type switch, but with an explicit
type rather than the type keyword:
and the result is a new value with the static type typeName .
That type must either be the concrete type held by the interface, or a second interface
type that the value can be converted to.
To extract the string we know is in the value, we could write:

But if it turns out that the value does not contain a string, the program will crash with a run-time error.
To guard against that, use the "comma, ok" idiom to test, safely, whether the value is a string:
If the type assertion fails, str will still exist and be of type string, but it will have
the zero value, an empty string.
As an illustration of the capability, here's an if - else
statement that's equivalent to the type switch that opened this section.

Contoh Kode / Perintah CLI:

```
type Stringer interface {
    String() string
}

var value interface{} // Value provided by caller.
switch str := value.(type) {
case string:
    return str
case Stringer:
    return str.String()
}

// ----------------------------------------

value.(typeName)

// ----------------------------------------

str := value.(string)
```

[golang_059] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Generality

Sumber: ht

Generality
If a type exists only to implement an interface and will
never have exported methods beyond that interface, there is
no need to export the type itself.
Exporting just the interface makes it clear the value has no
interesting behavior beyond what is described in the
interface.
It also avoids the need to repeat the documentation
on every instance of a common method.
In such cases, the constructor should return an interface value
rather than the implementing type.
As an example, in the hash libraries
both crc32.NewIEEE and adler32.New
return the interface type hash.Hash32 .
Substituting the CRC-32 algorithm for Adler-32 in a Go program
requires only changing the constructor call;
the rest of the code is unaffected by the change of algorithm.
A similar approach allows the streaming cipher algorithms
in the various crypto packages to be
separated from the block ciphers they chain together.
The Block interface
in the crypto/cipher package specifies the
behavior of a block cipher, which provides encryption
of a single block of data.

Then, by analogy with the bufio package,
cipher packages that implement this interface
can be used to construct streaming ciphers, represented
by the Stream interface, without
knowing the details of the block encryption.
The crypto/cipher interfaces look like this:
Here's the definition of the counter mode (CTR) stream,
which turns a block cipher into a streaming cipher; notice
that the block cipher's details are abstracted away:
NewCTR applies not
just to one specific encryption algorithm and data source but to any
implementation of the Block interface and any
Stream . Because they return
interface values, replacing CTR
encryption with other encryption modes is a localized change. The constructor
calls must be edit

Contoh Kode / Perintah CLI:

```
type Block interface {
    BlockSize() int
    Encrypt(dst, src []byte)
    Decrypt(dst, src []byte)
}

type Stream interface {
    XORKeyStream(dst, src []byte)
}

// ----------------------------------------

// NewCTR returns a Stream that encrypts/decrypts using the given Block in
// counter mode. The length of iv must be the same as the Block's block size.
func NewCTR(block Block, iv []byte) Stream
```

[golang_060] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Interfaces and methods

Sumber: ht

Interfaces and methods
Since almost anything can have methods attached, almost anything can
satisfy an interface. One illustrative example is in the http
package, which defines the Handler interface. Any object
that implements Handler can serve HTTP requests.
ResponseWriter is itself an interface that provides access
to the methods needed to return the response to the client.
Those methods include the standard Write method, so an
http.ResponseWriter can be used wherever an io.Writer
can be used.
Request is a struct containing a parsed representation
of the request from the client.
For brevity, let's ignore POSTs and assume HTTP requests are always
GETs; that simplification does not affect the way the handlers are set up.
Here's a trivial implementation of a handler to count the number of times
the page is visited.

(Keeping with our theme, note how Fprintf can print to an
http.ResponseWriter .)
In a real server, access to ctr.n would need protection from
concurrent access.
See the sync and atomic packages for suggestions.
For reference, here's how to attach such a server to a node on the URL tree.
But why make Counter a struct? An integer is all that's needed.
(The receiver needs to be a pointer so the increment is visible to the caller.)
What if your program has some internal state that needs to be notified that a page
has been visited? Tie a channel to the web page.
Finally, let's say we wanted to present on /args the arguments
used when invoking the server binary.
It's easy to write a function to print the arguments.
How do we turn that into an HTTP server? We could make ArgServer
a method of some type whose value we ignore, but there's a cleaner way.
Since we can define a method for any type except pointers and interfaces,
we can write a

Contoh Kode / Perintah CLI:

```
type Handler interface {
    ServeHTTP(ResponseWriter, *Request)
}

// --------------------------------------

// Simple counter server.
type Counter struct {
    n int
}

func (ctr *Counter) ServeHTTP(w http.ResponseWriter, req *http.Request) {
    ctr.n++
    fmt.Fprintf(w, "counter = %d\n", ctr.n)
}

// --------------------------------------

import "net/http"
...
ctr := new(Counter)
http.Handle("/counter", ctr)
```

[golang_061] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: The blank identifier

Sumber: ht

The blank identifier
We've mentioned the blank identifier a couple of times now, in the context of
for range loops
and maps .
The blank identifier can be assigned or declared with any value of any type, with the
value discarded harmlessly.
It's a bit like writing to the Unix /dev/null file:
it represents a write-only value
to be used as a place-holder

where a variable is needed but the actual value is irrelevant.
It has uses beyond those we've seen already.

[golang_062] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: The blank identifier in multiple assignment

Sumber: ht

The blank identifier in multiple assignment
The use of a blank identifier in a for range loop is a
special case of a general situation: multiple assignment.
If an assignment requires multiple values on the left side,
but one of the values will not be used by the program,
a blank identifier on the left-hand-side of
the assignment avoids the need
to create a dummy variable and makes it clear that the
value is to be discarded.
For instance, when calling a function that returns
a value and an error, but only the error is important,
use the blank identifier to discard the irrelevant value.
Occasionally you'll see code that discards the error value in order
to ignore the error; this is terrible practice. Always check error returns;
they're provided for a reason.

Contoh Kode / Perintah CLI:

```
if _, err := os.Stat(path); os.IsNotExist(err) {
    fmt.Printf("%s does not exist\n", path)
}

// ----------------------------------------

// Bad! This code will crash if path does not exist.
fi, _ := os.Stat(path)
if fi.IsDir() {
    fmt.Printf("%s is a directory\n", path)
}
```

[golang_063] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Unused imports and variables

Sumber: ht

Unused imports and variables
It is an error to import a package or to declare a variable without using it.
Unused imports bloat the program and slow compilation,
while a variable that is initialized but not used is at least
a wasted computation and perhaps indicative of a
larger bug.
When a program is under active development, however,
unused imports and variables often arise and it can
be annoying to delete them just to have the compilation proceed,
only to have them be needed again later.
The blank identifier provides a workaround.
This half-written program has two unused imports

(fmt and io)
and an unused variable (fd),
so it will not compile, but it would be nice to see if the
code so far is correct.
{{code "/doc/progs/eff_unused1.go" `/package/` `$`}}
To silence complaints about the unused imports, use a
blank identifier to refer to a symbol from the imported package.
Similarly, assigning the unused variable fd
to the blank identifier will silence the unused variable error.
This version of the program does compile.
{{code "/doc/progs/eff_unused2.go" `/package/` `$`}}
By convention, the global declarations to silence import errors
should come right after the imports and be commented,
both to make them easy to find and as a reminder to clean things up later.

[golang_064] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Import for side effect

Sumber: ht

Import for side effect
An unused import like fmt or io in the
previous example should eventually be used or removed:
blank assignments identify code as a work in progress.
But sometimes it is useful to import a package only for its
side effects, without any explicit use.
For example, during its init function,
the net/http/pprof
package registers HTTP handlers that provide
debugging information. It has an exported API, but
most clients need only the handler registration and
access the data through a web page.
To import the package only for its side effects, rename the package
to the blank identifier:
This form of import makes clear that the package is being
imported for its side effects, because there is no other possible
use of the package: in this file, it doesn't have a name.
(If it did, and we didn't use that name, the compiler would reject the program.)

Contoh Kode / Perintah CLI:

```
import _ "net/http/pprof"
```

[golang_065] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Interface checks

Sumber: ht

Interface checks
As we saw in the discussion of interfaces above,
a type need not declare explicitly that it implements an interface.
Instead, a type implements the interface just by implementing the interface's methods.
In practice, most interface conversions are static and therefore checked at compile time.
For example, passing an *os.File to a function
expecting an io.Reader will not compile unless

*os.File implements the io.Reader interface.
Some interface checks do happen at run-time, though.
One instance is in the encoding/json
package, which defines a Marshaler
interface. When the JSON encoder receives a value that implements that interface,
the encoder invokes the value's marshaling method to convert it to JSON
instead of doing the standard conversion.
The encoder checks this property at run time with a type assertion like:
If it's necessary only to ask whether a type implements an interface, without
actually using the interface itself, perhaps as part of an error check, use the blank
identifier to ignore the type-asserted value:
One place this situation arises is when it is necessary to guarantee within the package implementing the type that
it actually satisfies the interface.
If a type -- for example,
json.RawMessage -- needs
a custom JSON representation, it should implement
json.Marshaler , but there are no static conversions that would
cause the compiler to verify this automatically.
If the type inadvertently fails to satisfy the interface, the JSON encoder will still work,
but will not use the custom implementation.
To guarantee that the implementation is correct,
a global declaration using the blank identifier can be used in the package:
In this declaration, the assignment involving a conversion of a
*RawMessage to a Marshaler
requires that

Contoh Kode / Perintah CLI:

```
m, ok := val.(json.Marshaler)

// ----------------------------------------

if _, ok := val.(json.Marshaler); ok {
    fmt.Printf("value %v of type %T implements json.Marshaler\n", val, val)
}

// ----------------------------------------

var _ json.Marshaler = (*RawMessage)(nil)
```

[golang_066] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Embedding

Sumber: ht

Embedding
Go does not provide the typical, type-driven notion of subclassing,
but it does have the ability to “borrow” pieces of an
implementation by embedding types within a struct or
interface.
Interface embedding is very simple.
We've mentioned the io.Reader and io.Writer interfaces before;
here are their definitions.
The io package also exports several other interfaces
that specify objects that can implement several such methods.
For instance, there is io.ReadWriter , an interface

containing both Read and Write .
We could specify io.ReadWriter by listing the
two methods explicitly, but it's easier and more evocative
to embed the two interfaces to form the new one, like this:
This says just what it looks like: A ReadWriter can do
what a Reader does and what a Writer
does; it is a union of the embedded interfaces.
Only interfaces can be embedded within interfaces.
The same basic idea applies to structs, but with more far-reaching
implications. The bufio package has two struct types,
bufio.Reader and bufio.Writer , each of
which of course implements the analogous interfaces from package
io .
And bufio also implements a buffered reader/writer,
which it does by combining a reader and a writer into one struct
using embedding: it lists the types within the struct
but does not give them field names.
The embedded elements are pointers to structs and of course
must be initialized to point to valid structs before they
can be used.
The ReadWriter struct could be written as
but then to promote the methods of the fields and to
satisfy the io interfaces, we would also need
to provide forwarding methods, like this:
By embedding the structs directly, we avoid this bookkeeping.
The methods of embedded types come along for free, which means

Contoh Kode / Perintah CLI:

```
type Reader interface {
    Read(p []byte) (n int, err error)
}

type Writer interface {
    Write(p []byte) (n int, err error)
}

// ----------------------------------------

// ReadWriter is the interface that combines the Reader and Writer interfaces.
type ReadWriter interface {
    Reader
    Writer
}

// ----------------------------------------

// ReadWriter stores pointers to a Reader and a Writer.
// It implements io.ReadWriter.
type ReadWriter struct {
    *Reader  // *bufio.Reader
    *Writer  // *bufio.Writer
}
```

[golang_067] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Share by communicating

Sumber: ht

Share by communicating
Concurrent programming is a large topic and there is space only for some
Go-specific highlights here.
Concurrent programming in many environments is made difficult by the
subtleties required to implement correct access to shared variables. Go encourages
a different approach in which shared values are passed around on channels
and, in fact, never actively shared by separate threads of execution.
Only one goroutine has access to the value at any given time.
Data races cannot occur, by design.
To encourage this way of thinking we have reduced it to a slogan:
Do not communicate by sharing memory;
instead, share memory by communicating.
This approach can be taken too far. Reference counts may be best done
by putting a mutex around an integer variable, for instance. But as a
high-level approach, using channels to control access makes it easier
to write clear, correct programs.
One way to think about this model is to consider a typical single-threaded
program running on one CPU. It has no need for synchronization primitives.
Now run another such instance; it too needs no synchronization. Now let those
two communicate; if the communication is the synchronizer, there's still no need
for other synchronization. Unix pipelines, for example, fit this model
perfectly. Although Go's approach to concurrency originates in Hoare's
Communicating Sequential Processes (CSP),
it can also be seen as a type-safe generalization of Unix pipes.

[golang_068] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Goroutines

Sumber: ht

Goroutines
They're called goroutines because the existing
terms—threads, coroutines, processes, and so on—convey
inaccurate connotations. A goroutine has a simple model: it is a
function executing concurrently with other goroutines in the same
address space. It is lightweight, costing little more than the
allocation of stack space.
And the stacks start small, so they are cheap, and grow
by allocating (and freeing) heap storage as required.
Goroutines are multiplexed onto multiple OS threads so if one should
block, such as while waiting for I/O, others continue to run. Their
design hides many of the complexities of thread creation and
management.
Prefix a function or method call with the go
keyword to run the call in a new goroutine.
When the call completes, the goroutine
exits, silently. (The effect is similar to the Unix shell's
& notation for running a command in the
background.)

A function literal can be handy in a goroutine invocation.
In Go, function literals are closures: the implementation makes
sure the variables referred to by the function survive as long as they are active.
These examples aren't too practical because the functions have no way of signaling
completion. For that, we need channels.

Contoh Kode / Perintah CLI:

```
go list.Sort()  // run list.Sort concurrently; don't wait for it.

// ---------------------------------------

func Announce(message string, delay time.Duration) {
    go func() {
        time.Sleep(delay)
        fmt.Println(message)
    }()  // Note the parentheses - must call the function.
}
```

[golang_069] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Channels

Sumber: ht

Channels
Like maps, channels are allocated with make , and
the resulting value acts as a reference to an underlying data structure.
If an optional integer parameter is provided, it sets the buffer size for the channel.
The default is zero, for an unbuffered or synchronous channel.
Unbuffered channels combine communication—the exchange of a value—with
synchronization—guaranteeing that two calculations (goroutines) are in
a known state.
There are lots of nice idioms using channels. Here's one to get us started.
In the previous section we launched a sort in the background. A channel
can allow the launching goroutine to wait for the sort to complete.
Receivers always block until there is data to receive.
If the channel is unbuffered, the sender blocks until the receiver has
received the value.
If the channel has a buffer, the sender blocks only until the
value has been copied to the buffer; if the buffer is full, this
means waiting until some receiver has retrieved a value.
A buffered channel can be used like a semaphore, for instance to
limit throughput. In this example, incoming requests are passed
to handle , which sends a value into the channel, processes
the request, and then receives a value from the channel
to ready the “semaphore” for the next consumer.
The capacity of the channel buffer limits the number of
simultaneous calls to process .
Once MaxOutstanding handlers are executing process ,
any more will block trying to send into the filled channel buffer,
until one of the existing handlers finishes and receives from the buffer.
This design has a problem, though: Serve
creates a new goroutine for
every incoming request, even though only MaxOutstanding
of them can run at any moment.
As a result, the program can consume unlimit

Contoh Kode / Perintah CLI:

```
ci := make(chan int)            // unbuffered channel of integers
cj := make(chan int, 0)         // unbuffered channel of integers
cs := make(chan *os.File, 100)  // buffered channel of pointers to Files

// ----------------------------------------

c := make(chan int)  // Allocate a channel.
// Start the sort in a goroutine; when it completes, signal on the channel.
go func() {
    list.Sort()
    c <- 1  // Send a signal; value does not matter.
}()
doSomethingForAWhile()
<-c   // Wait for sort to finish; discard sent value.

// ----------------------------------------

var sem = make(chan int, MaxOutstanding)

func handle(r *Request) {
    sem <- 1    // Wait for active queue to drain.
    process(r)  // May take a long time.
    <-sem       // Done; enable next request to run.
}

func Serve(queue chan *Request) {
    for {
        req := <-queue
        go handle(req)  // Don't wait for handle to finish.
    }
}
```

[golang_070] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Channels of channels

Sumber: ht

Channels of channels
One of the most important properties of Go is that
a channel is a first-class value that can be allocated and passed
around like any other. A common use of this property is
to implement safe, parallel demultiplexing.
In the example in the previous section, handle was
an idealized handler for a request but we didn't define the
type it was handling. If that type includes a channel on which
to reply, each client can provide its own path for the answer.
Here's a schematic definition of type Request .
The client provides a function and its arguments, as well as
a channel inside the request object on which to receive the answer.
On the server side, the handler function is the only thing that changes.
There's clearly a lot more to do to make it realistic, but this
code is a framework for a rate-limited, parallel, non-blocking RPC
system, and there's not a mutex in sight.

Contoh Kode / Perintah CLI:

```
type Request struct {
    args        []int
```

```
    f           func([]int) int
    resultChan  chan int
}

// --------------------------------------

func sum(a []int) (s int) {
    for _, v := range a {
        s += v
    }
    return
}

request := &Request{[]int{3, 4, 5}, sum, make(chan int)}
// Send request
clientRequests <- request
// Wait for response.
fmt.Printf("answer: %d\n", <-request.resultChan)

// --------------------------------------

func handle(queue chan *Request) {
    for req := range queue {
        req.resultChan <- req.f(req.args)
    }
}
```

[golang_071] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Parallelization

Sumber: ht

Parallelization
Another application of these ideas is to parallelize a calculation
across multiple CPU cores. If the calculation can be broken into
separate pieces that can execute independently, it can be parallelized,
with a channel to signal when each piece completes.
Let's say we have an expensive operation to perform on a vector of items,
and that the value of the operation on each item is independent,
as in this idealized example.
We launch the pieces independently in a loop, one per CPU.
They can complete in any order but it doesn't matter; we just
count the completion signals by draining the channel after
launching all the goroutines.
Rather than create a constant value for numCPU, we can ask the runtime what
value is appropriate.
The function runtime.NumCPU
returns the number of hardware CPU cores in the machine, so we could write
There is also a function
runtime.GOMAXPROCS ,
which reports (or sets)
the user-specified number of cores that a Go program can have running
simultaneously.
It defaults to the value of runtime.NumCPU but can be
overridden by setting the similarly named shell environment variable
or by calling the function with a positive number. Calling it with
zero just queries the value.

Therefore if we want to honor the user's resource request, we should write
Be sure not to confuse the ideas of concurrency -- structuring a program
as independently executing components -- and parallelism -- executing
calculations in parallel for efficiency on multiple CPUs.
Although the concurrency features of Go can make some problems easy
to structure as parallel computations, Go is a concurrent language,
not a parallel one, and not all parallelization problems fit Go's model.
For a discussion of the distinction, see the talk cited in
this
blog post .

Contoh Kode / Perintah CLI:

```
type Vector []float64

// Apply the operation to v[i], v[i+1] ... up to v[n-1].
func (v Vector) DoSome(i, n int, u Vector, c chan int) {
    for ; i < n; i++ {
        v[i] += u.Op(v[i])
    }
    c <- 1    // signal that this piece is done
}

// ----------------------------------------

const numCPU = 4 // number of CPU cores

func (v Vector) DoAll(u Vector) {
    c := make(chan int, numCPU)  // Buffering optional but sensible.
    for i := 0; i < numCPU; i++ {
        go v.DoSome(i*len(v)/numCPU, (i+1)*len(v)/numCPU, u, c)
    }
    // Drain the channel.
    for i := 0; i < numCPU; i++ {
        <-c    // wait for one task to complete
    }
    // All done.
}

// ----------------------------------------

var numCPU = runtime.NumCPU()
```

[golang_072] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: A leaky buffer

Sumber: ht

A leaky buffer
The tools of concurrent programming can even make non-concurrent
ideas easier to express. Here's an example abstracted from an RPC
package. The client goroutine loops receiving data from some source,
perhaps a network. To avoid allocating and freeing buffers, it keeps
a free list, and uses a buffered channel to represent it. If the
channel is empty, a new buffer gets allocated.
Once the message buffer is ready, it's sent to the server on
serverChan .
The server loop receives each message from the client, processes it,
and returns the buffer to the free list.

The client attempts to retrieve a buffer from freeList ;
if none is available, it allocates a fresh one.
The server's send to freeList puts b back
on the free list unless the list is full, in which case the
buffer is dropped on the floor to be reclaimed by
the garbage collector.
(The default clauses in the select
statements execute when no other case is ready,
meaning that the selects never block.)
This implementation builds a leaky bucket free list
in just a few lines, relying on the buffered channel and
the garbage collector for bookkeeping.

Contoh Kode / Perintah CLI:

```
var freeList = make(chan *Buffer, 100)
var serverChan = make(chan *Buffer)

func client() {
    for {
        var b *Buffer
        // Grab a buffer if available; allocate if not.
        select {
        case b = <-freeList:
            // Got one; nothing more to do.
        default:
            // None free, so allocate a new one.
            b = new(Buffer)
        }
        load(b)              // Read next message from the net.
        serverChan <- b      // Send to server.
    }
}

// ----------------------------------------

func server() {
    for {
        b := <-serverChan    // Wait for work.
        process(b)
        // Reuse buffer if there's room.
        select {
        case freeList <- b:
            // Buffer on free list; nothing more to do.
        default:
            // Free list full, just carry on.
        }
    }
}
```

[golang_073] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Errors

Sumber: ht

Errors
Library routines must often return some sort of error indication to
the caller.
As mentioned earlier, Go's multivalue return makes it
easy to return a detailed error description alongside the normal

return value.
It is good style to use this feature to provide detailed error information.
For example, as we'll see, os.Open doesn't
just return a nil pointer on failure, it also returns an
error value that describes what went wrong.
By convention, errors have type error ,
a simple built-in interface.
A library writer is free to implement this interface with a
richer model under the covers, making it possible not only
to see the error but also to provide some context.
As mentioned, alongside the usual *os.File
return value, os.Open also returns an
error value.
If the file is opened successfully, the error will be nil ,
but when there is a problem, it will hold an
os.PathError :
PathError 's Error generates
a string like this:
Such an error, which includes the problematic file name, the
operation, and the operating system error it triggered, is useful even
if printed far from the call that caused it;
it is much more informative than the plain
"no such file or directory".
When feasible, error strings should identify their origin, such as by having
a prefix naming the operation or package that generated the error. For example, in package
image , the string representation for a decoding error due to an
unknown format is "image: unknown format".
Callers that care about the precise error details can
use a type switch or a type assertion to look for specific
errors and extract details. For PathErrors
this might include examining the internal Err
field for recoverable failures.
The second if statement here is another type assertion .
If it fails, ok will b

Contoh Kode / Perintah CLI:

```
type error interface {
    Error() string
}

// ---------------------------------------

// PathError records an error and the operation and
// file path that caused it.
type PathError struct {
    Op string    // "open", "unlink", etc.
    Path string  // The associated file.
    Err error    // Returned by the system call.
}

func (e *PathError) Error() string {
    return e.Op + " " + e.Path + ": " + e.Err.Error()
}

// ---------------------------------------
```

```
open /etc/passwx: no such file or directory
```

[golang_074] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Panic

Sumber: ht

Panic
The usual way to report an error to a caller is to return an
error as an extra return value. The canonical
Read method is a well-known instance; it returns a byte
count and an error . But what if the error is
unrecoverable? Sometimes the program simply cannot continue.
For this purpose, there is a built-in function panic
that in effect creates a run-time error that will stop the program
(but see the next section). The function takes a single argument
of arbitrary type—often a string—to be printed as the
program dies. It's also a way to indicate that something impossible has
happened, such as exiting an infinite loop.
This is only an example but real library functions should
avoid panic . If the problem can be masked or worked
around, it's always better to let things continue to run rather
than taking down the whole program. One possible counterexample
is during initialization: if the library truly cannot set itself up,
it might be reasonable to panic, so to speak.

Contoh Kode / Perintah CLI:

```
// A toy implementation of cube root using Newton's method.
func CubeRoot(x float64) float64 {
    z := x/3   // Arbitrary initial value
    for i := 0; i < 1e6; i++ {
        prevz := z
        z -= (z*z*z-x) / (3*z*z)
        if veryClose(z, prevz) {
            return z
        }
    }
    // A million iterations has not converged; something is wrong.
    panic(fmt.Sprintf("CubeRoot(%g) did not converge", x))
}

// ----------------------------------------

var user = os.Getenv("USER")

func init() {
    if user == "" {
        panic("no value for $USER")
    }
}
```

[golang_075] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: Recover

Sumber: ht

Recover
When panic is called, including implicitly for run-time
errors such as indexing a slice out of bounds or failing a type
assertion, it immediately stops execution of the current function
and begins unwinding the stack of the goroutine, running any deferred
functions along the way. If that unwinding reaches the top of the
goroutine's stack, the program dies. However, it is possible to
use the built-in function recover to regain control
of the goroutine and resume normal execution.
A call to recover stops the unwinding and returns the
argument passed to panic . Because the only code that
runs while unwinding is inside deferred functions, recover
is only useful inside deferred functions.
One application of recover is to shut down a failing goroutine
inside a server without killing the other executing goroutines.
In this example, if do(work) panics, the result will be
logged and the goroutine will exit cleanly without disturbing the
others. There's no need to do anything else in the deferred closure;
calling recover handles the condition completely.
Because recover always returns nil unless called directly
from a deferred function, deferred code can call library routines that themselves
use panic and recover without failing. As an example,
the deferred function in safelyDo might call a logging function before
calling recover , and that logging code would run unaffected
by the panicking state.
With our recovery pattern in place, the do
function (and anything it calls) can get out of any bad situation
cleanly by calling panic . We can use that idea to
simplify error handling in complex software. Let's look at an
idealized version of a regexp package, which reports
parsing errors by calling panic with a local
error type. Here's the

Contoh Kode / Perintah CLI:

```
func server(workChan <-chan *Work) {
    for work := range workChan {
        go safelyDo(work)
    }
}

func safelyDo(work *Work) {
    defer func() {
        if err := recover(); err != nil {
            log.Println("work failed:", err)
        }
    }()
    do(work)
}

// ---------------------------------------

// Error is the type of a parse error; it satisfies the error interface.
```

```
type Error string
func (e Error) Error() string {
    return string(e)
}

// error is a method of *Regexp that reports parsing errors by
// panicking with an Error.
func (regexp *Regexp) error(err string) {
    panic(Error(err))
}

// Compile returns a parsed representation of the regular expression.
func Compile(str string) (regexp *Regexp, err error) {
    regexp = new(Regexp)
    // doParse will panic if there is a parse error.
    defer func() {
        if e := recover(); e != nil {
            regexp = nil    // Clear return value.
            err = e.(Error) // Will re-panic if not a parse error.
        }
    }()
    return regexp.doParse(str), nil
}

// --------------------------------------

if pos == 0 {
    re.error("'*' illegal at start of expression")
}
```

[golang_076] Go - Effective Go - Syntax & Idioms

Effective Go: Core Syntax, Data, Methods, Concurrency & Web Server: A web server

Sumber: ht

A web server
Let's finish with a complete Go program, a web server.
This one is actually a kind of web re-server.
Google provides a service at chart.apis.google.com
that does automatic formatting of data into charts and graphs.
It's hard to use interactively, though,
because you need to put the data into the URL as a query.
The program here provides a nicer interface to one form of data: given a short piece of text,
it calls on the chart server to produce a QR code, a matrix of boxes that encode the
text.
That image can be grabbed with your cell phone's camera and interpreted as,
for instance, a URL, saving you typing the URL into the phone's tiny keyboard.
Here's the complete program.
An explanation follows.
{{code "/doc/progs/eff_qr.go" `/package/` `$`}}
The pieces up to main should be easy to follow.
The one flag sets a default HTTP port for our server. The template
variable templ is where the fun happens. It builds an HTML template
that will be executed by the server to display the page; more about
that in a moment.
The main function parses the flags and, using the mechanism
we talked about above, binds the function QR to the root path
for the server. Then http.ListenAndServe is called to start the

server; it blocks while the server runs.
QR just receives the request, which contains form data, and
executes the template on the data in the form value named s .
The template package html/template is powerful;
this program just touches on its capabilities.
In essence, it rewrites a piece of HTML text on the fly by substituting elements derived
from data items passed to templ.Execute , in this case the
form value.
Within the template text (templateStr),
double-brace-delimited pieces denote template actions.
The piece from {{"{{if .}}"}}
to {{"{{end}}

Contoh Kode / Perintah CLI:

```
verifying implementation
type Color uint32

// Check that Color implements image.Color and image.Image
var _ image.Color = Black
var _ image.Image = Black
```

[golang_077] Go - Modern Features - Go 1.22

Go 1.22 Release Notes (for loop scoping, range-over-integer, http.ServeMux routing): Introduction to Go 1.22

Sumber: ht

Introduction to Go 1.22
The latest Go release, version 1.22, arrives six months after Go 1.21 .
Most of its changes are in the implementation of the toolchain, runtime, and libraries.
As always, the release maintains the Go 1 promise of compatibility .
We expect almost all Go programs to continue to compile and run as before.

[golang_078] Go - Modern Features - Go 1.22

Go 1.22 Release Notes (for loop scoping, range-over-integer, http.ServeMux routing): Changes to the language

Sumber: ht

Changes to the language
Go 1.22 makes two changes to “for” loops.
Previously, the variables declared by a “for” loop were created once and updated by each iteration. In Go 1.22, each iteration of the loop creates new variables, to avoid accidental sharing bugs.
The transition support tooling
described in the proposal continues to work in the same way it did in Go 1.21.
“For” loops may now range over integers.
For example :
See the spec for details .
Go 1.22 includes a preview of a language change we are considering
for a future version of Go: range-over-function iterators .
Building with GOEXPERIMENT=rangefunc enables this feature.

Contoh Kode / Perintah CLI:

```
package main

import &amp;quot;fmt&amp;quot;
```

```
func main() {
  for i := range 10 {
    fmt.Println(10 - i)
  }
  fmt.Println(&amp;quot;go1.22 has lift-off!&amp;quot;)
}
```

[golang_079] Go - Modern Features - Go 1.22

Go 1.22 Release Notes (for loop scoping, range-over-integer, http.ServeMux routing): Go command

Sumber: ht

Go command
Commands in workspaces can now
use a vendor directory containing the dependencies of the
workspace. The directory is created by
go work vendor ,
and used by build commands when the -mod flag is set to
vendor , which is the default when a workspace vendor
directory is present.
Note that the vendor directory’s contents for a workspace are different
from those of a single module: if the directory at the root of a workspace also
contains one of the modules in the workspace, its vendor directory
can contain the dependencies of either the workspace or of the module,
but not both.
go get is no longer supported outside of a module in the
legacy GOPATH mode (that is, with GO111MODULE=off).
Other build commands, such as go build and
go test , will continue to work indefinitely
for legacy GOPATH programs.
go mod init no longer attempts to import
module requirements from configuration files for other vendoring tools
(such as Gopkg.lock).
go test -cover now prints coverage summaries
for covered packages that do not have their own test files. Prior to Go 1.22 a
go test -cover run for such a package would
report
? mymod/mypack [no test files]
and now with Go 1.22, functions in the package are treated as uncovered:
mymod/mypack coverage: 0.0% of statements
Note that if a package contains no executable code at all, we can’t report
a meaningful coverage percentage; for such packages the go tool
will continue to report that there are no test files.
go build commands that invoke the linker now error out if an
external (C) linker will be used but cgo is not enabled. (The Go runtime
requires cgo support to ensure that it is compatible with any additional
libraries added by the C linker.)

[golang_080] Go - Modern Features - Go 1.22

Go 1.22 Release Notes (for loop scoping, range-over-integer, http.ServeMux routing): Trace

Sumber: ht

Trace
The trace tool’s web UI has been gently refreshed as part of the
work to support the new tracer, resolving several issues and improving the
readability of various sub-pages.
The web UI now supports exploring traces in a thread-oriented view.
The trace viewer also now displays the full duration of all system calls.
These improvements only apply for viewing traces produced by programs built with
Go 1.22 or newer.
A future release will bring some of these improvements to traces produced by older
version of Go.

[golang_081] Go - Modern Features - Go 1.22

Go 1.22 Release Notes (for loop scoping, range-over-integer, http.ServeMux routing): Vet

Sumber: ht

Vet
References to loop variables
The behavior of the vet tool has changed to match
the new semantics (see above) of loop variables in Go 1.22.
When analyzing a file that requires Go 1.22 or newer
(due to its go.mod file or a per-file build constraint),
vet no longer reports references to
loop variables from within a function literal that
might outlive the iteration of the loop.
In Go 1.22, loop variables are created anew for each iteration,
so such references are no longer at risk of using a variable
after it has been updated by the loop.
New warnings for missing values after append
The vet tool now reports calls to
append that pass
no values to be appended to the slice, such as slice = append(slice) .
Such a statement has no effect, and experience has shown that is nearly always a mistake.
New warnings for deferring time.Since
The vet tool now reports a non-deferred call to
time.Since(t) within a defer statement.
This is equivalent to calling time.Now().Sub(t) before the defer statement,
not when the deferred function is called. In nearly all cases, the correct code
requires deferring the time.Since call. For example:
New warnings for mismatched key-value pairs in log/slog calls
The vet tool now reports invalid arguments in calls to functions and methods
in the structured logging package, log/slog ,
that accept alternating key/value pairs.
It reports calls where an argument in a key position is neither a
string nor a slog.Attr , and where a final key is missing its value.

Contoh Kode / Perintah CLI:

```
t := time.Now()
```

```
defer log.Println(time.Since(t)) // non-deferred call to time.Since
tmp := time.Since(t); defer log.Println(tmp) // equivalent to the previous defer

defer func() {
  log.Println(time.Since(t)) // a correctly deferred call to time.Since
}()
```

[golang_082] Go - Modern Features - Go 1.22

Go 1.22 Release Notes (for loop scoping, range-over-integer, http.ServeMux routing): Runtime

Sumber: ht

Runtime
The runtime now keeps type-based garbage collection metadata nearer to each heap object, improving the CPU performance (latency or throughput) of Go programs by 1 - 3%.
This change also reduces the memory overhead of the majority Go programs by approximately 1% by deduplicating redundant metadata.
Some programs may see a smaller improvement because this change adjusts the size class boundaries of the memory allocator, so some objects may be moved up a size class.
A consequence of this change is that some objects’ addresses that were previously always aligned to a 16 byte (or higher) boundary will now only be aligned to an 8 byte boundary.
Some programs that use assembly instructions that require memory addresses to be more than 8-byte aligned and rely on the memory allocator’s previous alignment behavior may break, but we expect such programs to be rare.
Such programs may be built with GOEXPERIMENT=noallocheaders to revert to the old metadata layout and restore the previous alignment behavior, but package owners should update their assembly code to avoid the alignment assumption, as this workaround will be removed in a future release.
On the windows/amd64 port , programs linking or loading Go libraries built with -buildmode=c-archive or -buildmode=c-shared can now use the SetUnhandledExceptionFilter Win32 function to catch exceptions not handled by the Go runtime. Note that this was already supported on the windows/386 port.

[golang_083] Go - Modern Features - Go 1.22

Go 1.22 Release Notes (for loop scoping, range-over-integer, http.ServeMux routing): Compiler

Sumber: ht

Compiler
Profile-guided Optimization (PGO) builds can now devirtualize a higher proportion of calls than previously possible.
Most programs from a representative set of Go programs now see between 2 and 14% improvement at runtime from enabling PGO.
The compiler now interleaves devirtualization and inlining, so interface method calls are better optimized.
Go 1.22 also includes a preview of an enhanced implementation of the compiler’s inlining phase that uses heuristics to boost inlinability at call sites deemed “important” (for example, in loops) and discourage inlining at call sites deemed “unimportant” (for example, on panic paths).
Building with GOEXPERIMENT=newinliner enables the new call-site heuristics; see issue #61502 for

more info and to provide feedback.

[golang_084] Go - Modern Features - Go 1.22

Go 1.22 Release Notes (for loop scoping, range-over-integer, http.ServeMux routing): Linker

Sumber: ht

Linker
The linker’s -s and -w flags are now behave more
consistently across all platforms.
The -w flag suppresses DWARF debug information generation.
The -s flag suppresses symbol table generation.
The -s flag also implies the -w flag, which can be
negated with -w=0 .
That is, -s -w=0 will generate a binary with DWARF
debug information generation but without the symbol table.
On ELF platforms, the -B linker flag now accepts a special form:
with -B gobuildid , the linker will generate a GNU
build ID (the ELF NT_GNU_BUILD_ID note) derived from the Go
build ID.
On Windows, when building with -linkmode=internal , the linker now
preserves SEH information from C object files by copying the .pdata
and .xdata sections into the final binary.
This helps with debugging and profiling binaries using native tools, such as WinDbg.
Note that until now, C functions’ SEH exception handlers were not being honored,
so this change may cause some programs to behave differently.
-linkmode=external is not affected by this change, as external linkers
already preserve SEH information.

[golang_085] Go - Modern Features - Go 1.22

Go 1.22 Release Notes (for loop scoping, range-over-integer, http.ServeMux routing): Bootstrap

Sumber: ht

Bootstrap
As mentioned in the Go 1.20 release notes , Go 1.22 now requires
the final point release of Go 1.20 or later for bootstrap.
We expect that Go 1.24 will require the final point release of Go 1.22 or later for bootstrap.

[golang_086] Go - Modern Features - Go 1.22

Go 1.22 Release Notes (for loop scoping, range-over-integer, http.ServeMux routing): New math/rand/v2 package

Sumber: ht

New math/rand/v2 package
Go 1.22 includes the first "v2" package in the standard library,
math/rand/v2 .
The changes compared to math/rand are
detailed in proposal #61716 . The most important changes are:
The Read method, deprecated in math/rand ,
was not carried forward for math/rand/v2 .
(It remains available in math/rand .)

The vast majority of calls to Read should use
crypto/rand 's Read instead.
Otherwise a custom Read can be constructed using the Uint64 method.
The global generator accessed by top-level functions is unconditionally randomly seeded.
Because the API guarantees no fixed sequence of results,
optimizations like per-thread random generator states are now possible.
The Source
interface now has a single Uint64 method;
there is no Source64 interface.
Many methods now use faster algorithms that were not possible to adopt in math/rand
because they changed the output streams.
The
Intn ,
Int31 ,
Int31n ,
Int63 ,
and
Int64n
top-level functions and methods from math/rand
are spelled more idiomatically in math/rand/v2 :
IntN ,
Int32 ,
Int32N ,
Int64 ,
and
Int64N .
There are also new top-level functions and methods
Uint32 ,
Uint32N ,
Uint64 ,
Uint64N ,
and
UintN .
The
new generic function N
is like
Int64N or
Uint64N
but works for any integer type.
For example a random duration from 0 up to 5 minutes is
rand.N(5*time.Minute) .
The Mitchell & Reeds LFSR generator provided by
math/rand 's Source
has been replaced by two more modern pseudo-random generator sources:
ChaCha8 and
PCG .
ChaCha8 is a new, cryptographically strong random number generator
roughly similar to PCG in efficiency.
ChaCha8 is the algorithm used for the top-level functions in math/rand/v2 .
As of Go 1.22, math/rand ’s top-level functions (when not explic

[golang_087] Go - Modern Features - Go 1.22

Go 1.22 Release Notes (for loop scoping, range-over-integer, http.ServeMux routing): New go/version package

Sumber: ht

New go/version package
The new go/version package implements functions
for validating and comparing Go version strings.

[golang_088] Go - Modern Features - Go 1.22

Go 1.22 Release Notes (for loop scoping, range-over-integer, http.ServeMux routing): Enhanced routing patterns

Sumber: ht

Enhanced routing patterns
HTTP routing in the standard library is now more expressive.
The patterns used by net/http.ServeMux have been enhanced to accept methods and wildcards.
Registering a handler with a method, like "POST /items/create" , restricts
invocations of the handler to requests with the given method. A pattern with a method takes precedence over a matching pattern without one.
As a special case, registering a handler with "GET" also registers it with "HEAD" .
Wildcards in patterns, like /items/{id} , match segments of the URL path.
The actual segment value may be accessed by calling the Request.PathValue method.
A wildcard ending in “…”, like /files/{path...} , must occur at the end of a pattern and matches all the remaining segments.
A pattern that ends in “/” matches all paths that have it as a prefix, as always.
To match the exact pattern including the trailing slash, end it with {$} ,
as in /exact/match/{$} .
If two patterns overlap in the requests that they match, then the more specific pattern takes precedence.
If neither is more specific, the patterns conflict.
This rule generalizes the original precedence rules and maintains the property that the order in which
patterns are registered does not matter.
This change breaks backwards compatibility in small ways, some obvious -- patterns with “{” and “}” behave differently --
and some less so -- treatment of escaped paths has been improved.
The change is controlled by a GODEBUG field named httpmuxgo121 .
Set httpmuxgo121=1 to restore the old behavior.

[golang_089] Go - Modern Features - Go 1.22

Go 1.22 Release Notes (for loop scoping, range-over-integer, http.ServeMux routing): Minor changes to the library

Sumber: ht

Minor changes to the library
As always, there are various minor changes and updates to the library,
made with the Go 1 promise of compatibility
in mind.
There are also various performance improvements, not enumerated here.
archive/tar
The new method Writer.AddFS adds all of the files from an fs.FS to the archive.
archive/zip
The new method Writer.AddFS adds all of the files from an fs.FS to the archive.
bufio

When a SplitFunc returns ErrFinalToken with a nil token, Scanner will now stop immediately.
Previously, it would report a final empty token before stopping, which was usually not desired.
Callers that do want to report a final empty token can do so by returning []byte{} rather than nil .
cmp
The new function Or returns the first in a sequence of values that is not the zero value.
crypto/tls
ConnectionState.ExportKeyingMaterial will now
return an error unless TLS 1.3 is in use, or the extended_master_secret extension is supported by both the server and client. crypto/tls has supported this extension since Go 1.20. This can be disabled with the
tlsunsafeekm=1 GODEBUG setting.
By default, the minimum version offered by crypto/tls servers is now TLS 1.2 if not specified with
config.MinimumVersion , matching the behavior of crypto/tls
clients. This change can be reverted with the tls10server=1 GODEBUG setting.
By default, cipher suites without ECDHE support are no longer offered by either clients or servers during pre-TLS 1.3 handshakes. This change can be reverted with the tlsrsakex=1 GODEBUG setting.
crypto/x509
The new CertPool.AddCertWithConstraint
method can be used to add customized constraints to root certificates to be applied during chain building.
On Android, root certificates will now be loaded from /data/

[golang_090] Go - Modern Features - Go 1.22

Go 1.22 Release Notes (for loop scoping, range-over-integer, http.ServeMux routing): Darwin

Sumber: ht

Darwin
On macOS on 64-bit x86 architecture (the darwin/amd64 port),
the Go toolchain now generates position-independent executables (PIE) by default.
Non-PIE binaries can be generated by specifying the -buildmode=exe
build flag.
On 64-bit ARM-based macOS (the darwin/arm64 port),
the Go toolchain already generates PIE by default.
Go 1.22 is the last release that will run on macOS 10.15 Catalina. Go 1.23 will require macOS 11 Big Sur or later.

[golang_091] Go - Modern Features - Go 1.22

Go 1.22 Release Notes (for loop scoping, range-over-integer, http.ServeMux routing): ARM

Sumber: ht

ARM
The GOARM environment variable now allows you to select whether to use software or hardware floating point.
Previously, valid GOARM values were 5 , 6 , or 7 . Now those same values can
be optionally followed by ,softfloat or ,hardfloat to select the floating-point implementation.
This new option defaults to softfloat for version 5 and hardfloat for versions
6 and 7 .

[golang_092] Go - Modern Features - Go 1.22

Go 1.22 Release Notes (for loop scoping, range-over-integer, http.ServeMux routing): Loong64

Sumber: ht

Loong64
The loong64 port now supports passing function arguments and results using registers.
The linux/loong64 port now supports the address sanitizer, memory sanitizer, new-style linker relocations, and the plugin build mode.

[golang_093] Go - Modern Features - Go 1.22

Go 1.22 Release Notes (for loop scoping, range-over-integer, http.ServeMux routing): OpenBSD

Sumber: ht

OpenBSD
Go 1.22 adds an experimental port to OpenBSD on big-endian 64-bit PowerPC
(openbsd/ppc64).
Why Go
Use Cases
Case Studies
Get Started
Playground
Tour
Stack Overflow
Help
Packages
Standard Library
About Go Packages
pkg.go.dev API
About
Download
Blog
Issue Tracker
Release Notes
Brand Guidelines
Code of Conduct
Connect
Bluesky
Mastodon
Twitter
GitHub
Slack
r/golang
Meetup
Golang Weekly
Opens in new window.
Copyright
Terms of Service
Privacy Policy
Report an Issue
go.dev uses cookies from Google to deliver and enhance the quality of its services and to

analyze traffic. Learn more.
Okay

[golang_094] Go - Modern Features - Go 1.23

Go 1.23 Release Notes (Range-over-function iterators iter.Seq, Tooling): Introduction to Go 1.23

Sumber: ht

Introduction to Go 1.23
The latest Go release, version 1.23, arrives six months after Go 1.22 .
Most of its changes are in the implementation of the toolchain, runtime, and libraries.
As always, the release maintains the Go 1 promise of compatibility .
We expect almost all Go programs to continue to compile and run as before.

[golang_095] Go - Modern Features - Go 1.23

Go 1.23 Release Notes (Range-over-function iterators iter.Seq, Tooling): Changes to the language

Sumber: ht

Changes to the language
The “range” clause in a “for-range” loop now accepts iterator functions of the following types as range expressions.
Calls of the iterator argument function produce the iteration values for the “for-range” loop.
For details see the iter package documentation, the
language spec , and the Range over Function
Types blog post .
For motivation see the 2022 “range-over-func” discussion .
Go 1.23 includes preview support for generic type aliases .
Building the toolchain with GOEXPERIMENT=aliastypeparams enables this feature within a package.
(Using generic alias types across package boundaries is not yet supported.)

Contoh Kode / Perintah CLI:

```
func(func() bool)
func(func(K) bool)
func(func(K, V) bool)
```

[golang_096] Go - Modern Features - Go 1.23

Go 1.23 Release Notes (Range-over-function iterators iter.Seq, Tooling): Telemetry

Sumber: ht

Telemetry
Starting in Go 1.23, the Go toolchain can collect usage and breakage
statistics that help the Go team understand how the Go toolchain is
used and how well it is working. We refer to these statistics as
Go telemetry .
Go telemetry is an opt-in system , controlled by the
go telemetry command .
By default, the toolchain programs
collect statistics in counter files that can be inspected locally
but are otherwise unused (go telemetry local).
To help us keep Go working well and understand Go usage,
please consider opting in to Go telemetry by running
go telemetry on .
In that mode,

anonymous counter reports are uploaded to
telemetry.go.dev weekly,
where they are aggregated into graphs and also made
available for download by any Go contributors or users
wanting to analyze the data.
See " Go Telemetry " for more details
about the Go Telemetry system.

[golang_097] Go - Modern Features - Go 1.23

Go 1.23 Release Notes (Range-over-function iterators iter.Seq, Tooling): Go command

Sumber: ht

Go command
Setting the GOROOT_FINAL environment variable no longer has an effect
(#62047).
Distributions that install the go command to a location other than
$GOROOT/bin/go should install a symlink instead of relocating
or copying the go binary.
The new go env -changed flag causes the command to print only
those settings whose effective value differs from the default value
that would be obtained in an empty environment with no prior uses of the -w flag.
The new go mod tidy -diff flag causes the command not to modify
the files but instead print the necessary changes as a unified diff.
It exits with a non-zero code if updates are needed.
The go list -m -json command now includes new Sum and GoModSum fields.
This is similar to the existing behavior of the go mod download -json command.
The new godebug directive in go.mod and go.work declares a
GODEBUG setting to apply for the work module or workspace in use.

[golang_098] Go - Modern Features - Go 1.23

Go 1.23 Release Notes (Range-over-function iterators iter.Seq, Tooling): Vet

Sumber: ht

Vet
The go vet subcommand now includes the
stdversion
analyzer, which flags references to symbols that are too new for the version
of Go in effect in the referring file. (The effective version is determined
by the go directive in the file’s enclosing go.mod file, and
by any //go:build constraints
in the file.)
For example, it will report a diagnostic for a reference to the
reflect.TypeFor function (introduced in go1.22) from a file in a
module whose go.mod file specifies go 1.21 .

[golang_099] Go - Modern Features - Go 1.23

Go 1.23 Release Notes (Range-over-function iterators iter.Seq, Tooling): Cgo

Sumber: ht

Cgo
cmd/cgo supports the new -ldflags flag for passing flags to the C linker.
The go command uses it automatically, avoiding “argument list too long”
errors with a very large CGO_LDFLAGS .

[golang_100] Go - Modern Features - Go 1.23

Go 1.23 Release Notes (Range-over-function iterators iter.Seq, Tooling): Trace

Sumber: ht

Trace
The trace tool now better tolerates partially broken traces by attempting to
recover what trace data it can. This functionality is particularly helpful when
viewing a trace that was collected during a program crash, since the trace data
leading up to the crash will now be recoverable under most
circumstances.

[golang_101] Go - Modern Features - Go 1.23

Go 1.23 Release Notes (Range-over-function iterators iter.Seq, Tooling): Runtime

Sumber: ht

Runtime
The traceback printed by the runtime after an unhandled panic or other
fatal error now indents the second and subsequent lines of the error
message (for example, the argument to panic) by a single tab, so that
it can be unambiguously distinguished from the stack trace of the
first goroutine. See #64590 for discussion.

[golang_102] Go - Modern Features - Go 1.23

Go 1.23 Release Notes (Range-over-function iterators iter.Seq, Tooling): Compiler

Sumber: ht

Compiler
The build time overhead to building with Profile Guided Optimization has been reduced significantly.
Previously, large builds could see 100%+ build time increase from enabling PGO.
In Go 1.23, overhead should be in the single digit percentages.
The compiler in Go 1.23 can now overlap the stack frame slots of local variables
accessed in disjoint regions of a function, which reduces stack usage
for Go applications.
For 386 and amd64, the compiler will use information from PGO to align certain
hot blocks in loops. This improves performance an additional 1-1.5% at
a cost of an additional 0.1% text and binary size. This is currently only implemented
on 386 and amd64 because it has not shown an improvement on other platforms.
Hot block alignment can be disabled with -gcflags=[<packages>=]-d=alignhot=0 .

[golang_103] Go - Modern Features - Go 1.23

Go 1.23 Release Notes (Range-over-function iterators iter.Seq, Tooling): Linker

Sumber: ht

Linker
The linker now disallows using a //go:linkname directive to refer to
internal symbols in the standard library (including the runtime) that
are not marked with //go:linkname on their definitions.
Similarly, the linker disallows references to such symbols from assembly
code.
For backward compatibility, existing usages of //go:linkname found in
a large open-source code corpus remain supported.
Any new references to standard library internal symbols will be disallowed.
A linker command line flag -checklinkname=0 can be used to disable
this check, for debugging and experimenting purposes.
When building a dynamically linked ELF binary (including PIE binary), the
new -bindnow flag enables immediate function binding.

[golang_104] Go - Modern Features - Go 1.23

Go 1.23 Release Notes (Range-over-function iterators iter.Seq, Tooling): Timer changes

Sumber: ht

Timer changes
Go 1.23 makes two significant changes to the implementation of
time.Timer and time.Ticker .
First, Timer s and Ticker s that are no longer referred to by the program
become eligible for garbage collection immediately, even if their
Stop methods have not been called.
Earlier versions of Go did not collect unstopped Timer s until after
they had fired and never collected unstopped Ticker s.
Second, the timer channel associated with a Timer or Ticker is
now unbuffered, with capacity 0.
The main effect of this change is that Go now guarantees
that for any call to a Reset or Stop method, no stale values
prepared before that call will be sent or received after the call.
Earlier versions of Go used channels with a one-element buffer,
making it difficult to use Reset and Stop correctly.
A visible effect of this change is that len and cap of timer channels
now returns 0 instead of 1, which may affect programs that
poll the length to decide whether a receive on the timer channel
will succeed.
Such code should use a non-blocking receive instead.
These new behaviors are only enabled when the main Go program
is in a module with a go.mod go line using Go 1.23.0 or later.
When Go 1.23 builds older programs, the old behaviors remain in effect.
The new GODEBUG setting asynctimerchan=1
can be used to revert back to asynchronous channel behaviors
even when a program names Go 1.23.0 or later in its go.mod file.

[golang_105] Go - Modern Features - Go 1.23

Go 1.23 Release Notes (Range-over-function iterators iter.Seq, Tooling): New unique package

Sumber: ht

New unique package
The new unique package provides facilities for
canonicalizing values (like “interning” or “hash-consing”).
Any value of comparable type may be canonicalized with the new
Make[T] function, which produces a reference to a canonical copy of
the value in the form of a Handle[T] .
Two Handle[T] are equal if and only if the values used to produce the
handles are equal, allowing programs to deduplicate values and reduce
their memory footprint.
Comparing two Handle[T] values is efficient, reducing down to a simple
pointer comparison.

[golang_106] Go - Modern Features - Go 1.23

Go 1.23 Release Notes (Range-over-function iterators iter.Seq, Tooling): Iterators

Sumber: ht

Iterators
The new iter package provides the basic definitions for working with
user-defined iterators.
The slices package adds several functions that work with iterators:
All returns an iterator over slice indexes and values.
Values returns an iterator over slice elements.
Backward returns an iterator that loops over
a slice backward.
Collect collects values from an iterator into
a new slice.
AppendSeq appends values from an iterator to
an existing slice.
Sorted collects values from an iterator into a
new slice, and then sorts the slice.
SortedFunc is like Sorted but with a
comparison function.
SortedStableFunc is like SortFunc
but uses a stable sort algorithm.
Chunk returns an iterator over consecutive
sub-slices of up to n elements of a slice.
The maps package adds several functions that work with iterators:
All returns an iterator over key-value pairs from a map.
Keys returns an iterator over keys in a map.
Values returns an iterator over values in a map.
Insert adds the key-value pairs from an iterator to an existing map.
Collect collects key-value pairs from an iterator into a new map and returns it.

[golang_107] Go - Modern Features - Go 1.23

Go 1.23 Release Notes (Range-over-function iterators iter.Seq, Tooling): New structs package

Sumber: ht

New structs package
The new structs package provides
types for struct fields that modify properties of
the containing struct type such as memory layout.
In this release, the only such type is
HostLayout
which indicates that a structure with a field of that
type has a layout that conforms to host platform
expectations. HostLayout should be used in types that
are passed to, returned from, or accessed
via a pointer passed to/from host APIs.
Without this marker, struct layout order is not
guaranteed by the language spec, though as of Go 1.23
the host and language layouts happen to match.

[golang_108] Go - Modern Features - Go 1.23

Go 1.23 Release Notes (Range-over-function iterators iter.Seq, Tooling): Minor changes to the library

Sumber: ht

Minor changes to the library
archive/tar
If the argument to FileInfoHeader implements the new FileInfoNames
interface, then the interface methods will be used to set the Uname/Gname
of the file header. This allows applications to override the system-dependent
Uname/Gname lookup.
crypto/tls
The TLS client now supports the Encrypted Client Hello draft specification .
This feature can be enabled by setting the Config.EncryptedClientHelloConfigList
field to an encoded ECHConfigList for the host that is being connected to.
The QUICConn type used by QUIC implementations includes new events
reporting on the state of session resumption, and provides a way for
the QUIC layer to add data to session tickets and session cache entries.
3DES cipher suites were removed from the default list used when
Config.CipherSuites is nil. The default can be reverted by adding tls3des=1 to
the GODEBUG environment variable.
The experimental post-quantum key exchange mechanism X25519Kyber768Draft00
is now enabled by default when Config.CurvePreferences is nil.
The default can be reverted by adding tlskyber=0 to the GODEBUG environment variable.
This can be useful when dealing with buggy TLS servers that do not handle large records correctly,
causing a timeout during the handshake (see TLS post-quantum TL;DR fail).
Go 1.23 changed the behavior of X509KeyPair and LoadX509KeyPair
to populate the Certificate.Leaf field of the returned Certificate .
The new x509keypairleaf GODEBUG setting is added for this behavior.
crypto/x509
CreateCertificateRequest now correctly supports RSA-PSS signature algorithms.
CreateCertificateRequest and CreateRevocationList now verify the generated signature using the signer's public

key. If the signature is invalid, an erro

[golang_109] Go - Modern Features - Go 1.23

Go 1.23 Release Notes (Range-over-function iterators iter.Seq, Tooling): Darwin

Sumber: ht

Darwin
As announced in the Go 1.22 release notes,
Go 1.23 requires macOS 11 Big Sur or later;
support for previous versions has been discontinued.

[golang_110] Go - Modern Features - Go 1.23

Go 1.23 Release Notes (Range-over-function iterators iter.Seq, Tooling): Linux

Sumber: ht

Linux
Go 1.23 is the last release that requires Linux kernel version 2.6.32 or later. Go 1.24 will require Linux kernel version 3.2 or later.

[golang_111] Go - Modern Features - Go 1.23

Go 1.23 Release Notes (Range-over-function iterators iter.Seq, Tooling): OpenBSD

Sumber: ht

OpenBSD
Go 1.23 adds experimental support for OpenBSD on 64-bit RISC-V (GOOS=openbsd , GOARCH=riscv64).

[golang_112] Go - Modern Features - Go 1.23

Go 1.23 Release Notes (Range-over-function iterators iter.Seq, Tooling): ARM64

Sumber: ht

ARM64
Go 1.23 introduces a new GOARM64 environment variable, which specifies the minimum target version of the ARM64 architecture at compile time. Allowed values are v8.{0-9} and v9.{0-5} . This may be followed by an option specifying extensions implemented by target hardware. Valid options are ,lse and ,crypto .
The GOARM64 environment variable defaults to v8.0 .

[golang_113] Go - Modern Features - Go 1.23

Go 1.23 Release Notes (Range-over-function iterators iter.Seq, Tooling): RISC-V

Sumber: ht

RISC-V
Go 1.23 introduces a new GORISCV64 environment variable, which selects the RISC-V user-mode application profile for which to compile. Allowed values are rva20u64 and rva22u64 .
The GORISCV64 environment variable defaults to rva20u64 .

[golang_114] Go - Modern Features - Go 1.23

Go 1.23 Release Notes (Range-over-function iterators iter.Seq, Tooling): Wasm

Sumber: ht

Wasm
The go_wasip1_wasm_exec script in GOROOT/misc/wasm has dropped support
for versions of wasmtime < 14.0.0.
Why Go
Use Cases

Case Studies
Get Started
Playground
Tour
Stack Overflow
Help
Packages
Standard Library
About Go Packages
pkg.go.dev API
About
Download
Blog
Issue Tracker
Release Notes
Brand Guidelines
Code of Conduct
Connect
Bluesky
Mastodon
Twitter
GitHub
Slack
r/golang
Meetup
Golang Weekly
Opens in new window.
Copyright
Terms of Service
Privacy Policy
Report an Issue
go.dev uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic. Learn more.
Okay